

Redes Neurais Recorrentes e Tratamento de Sequências

Prof. Sérgio Canuto - sergio.canuto@ifg.edu.br

Redes Recorrentes

INSTITUTO FEDERAL
Goiás

- Contextualização
- Motivação/Justificativa
- Problemas com Redes Neurais Recorrentes (RNN)
- Tipos de RNN
- Implementação de RNNs

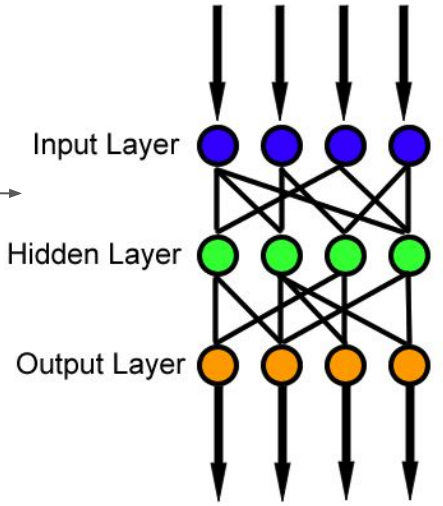


INSTITUTO FEDERAL
Goiás

Contextualização

- Até o momento:
  - Tratamos uma única entrada (ex. um conjunto de features, uma imagem, etc.)
  - Tratamos redes neurais diretas (feedforward networks)

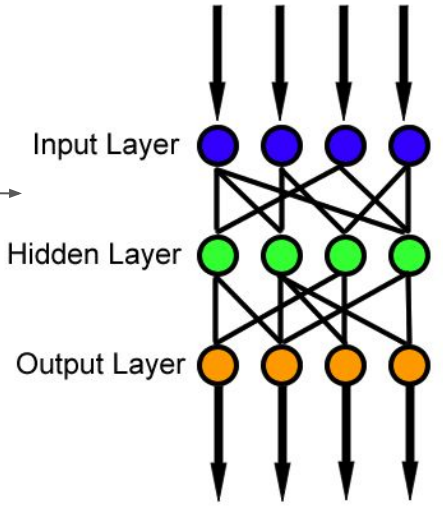
Rede Neural Direta (feedforward)



Contextualização

- Até o momento:
  - Tratamos uma única entrada (ex. um conjunto de features, uma imagem, etc.)
  - Tratamos redes neurais diretas (feedforward networks)

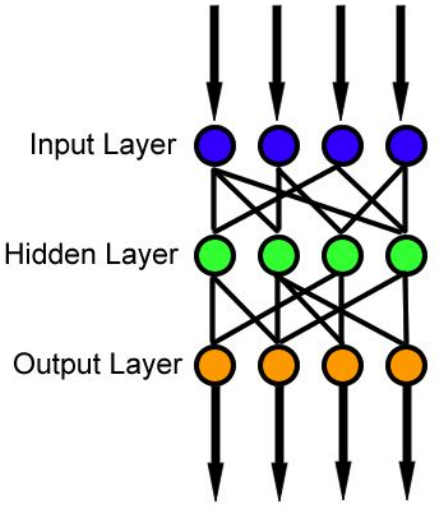
Rede Neural Direta (feedforward)



Contextualização

- Até o momento:
  - Tratamos uma única entrada (ex. um conjunto de features, uma imagem, etc.)
  - Tratamos redes neurais diretas (feedforward ~~networks~~)
- só que…
  - Muitas entradas são **sequenciais** ou precisam receber sinais dependentes do tempo, ex.:
    - Dados de sensores (velocidade, temperatura, consumo energético, etc)
    - Texto (texto=sequência de palavras)
    - Fala/música (reconhecimento de fala, classificação de música)
    - Vídeos (detecção de objetos/faces)
  - "Snapshots" podem não ser informativos (ex: trajetória de uma bola)
- **Contexto temporal** é importante!

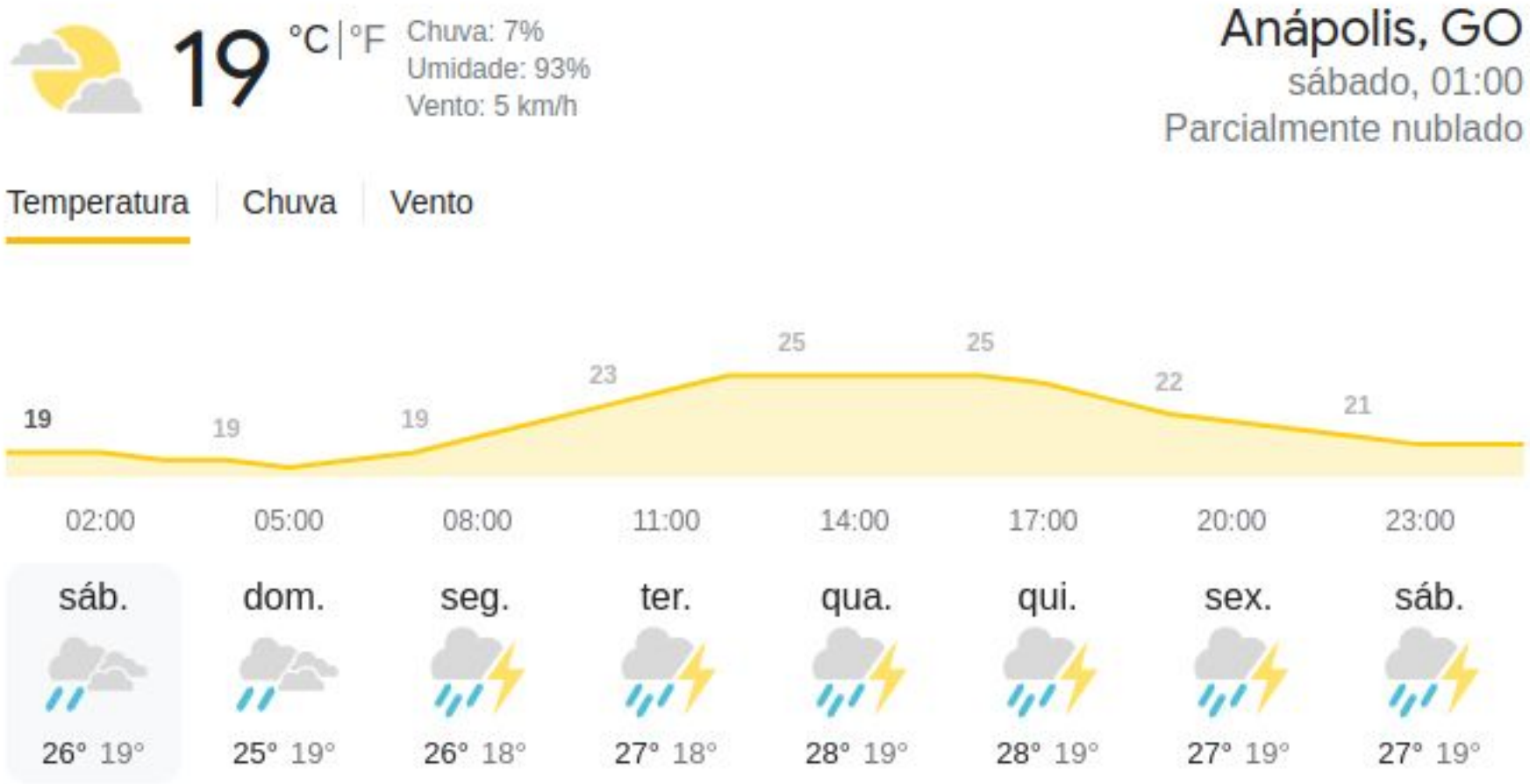
Rede Neural Direta (feedforward)



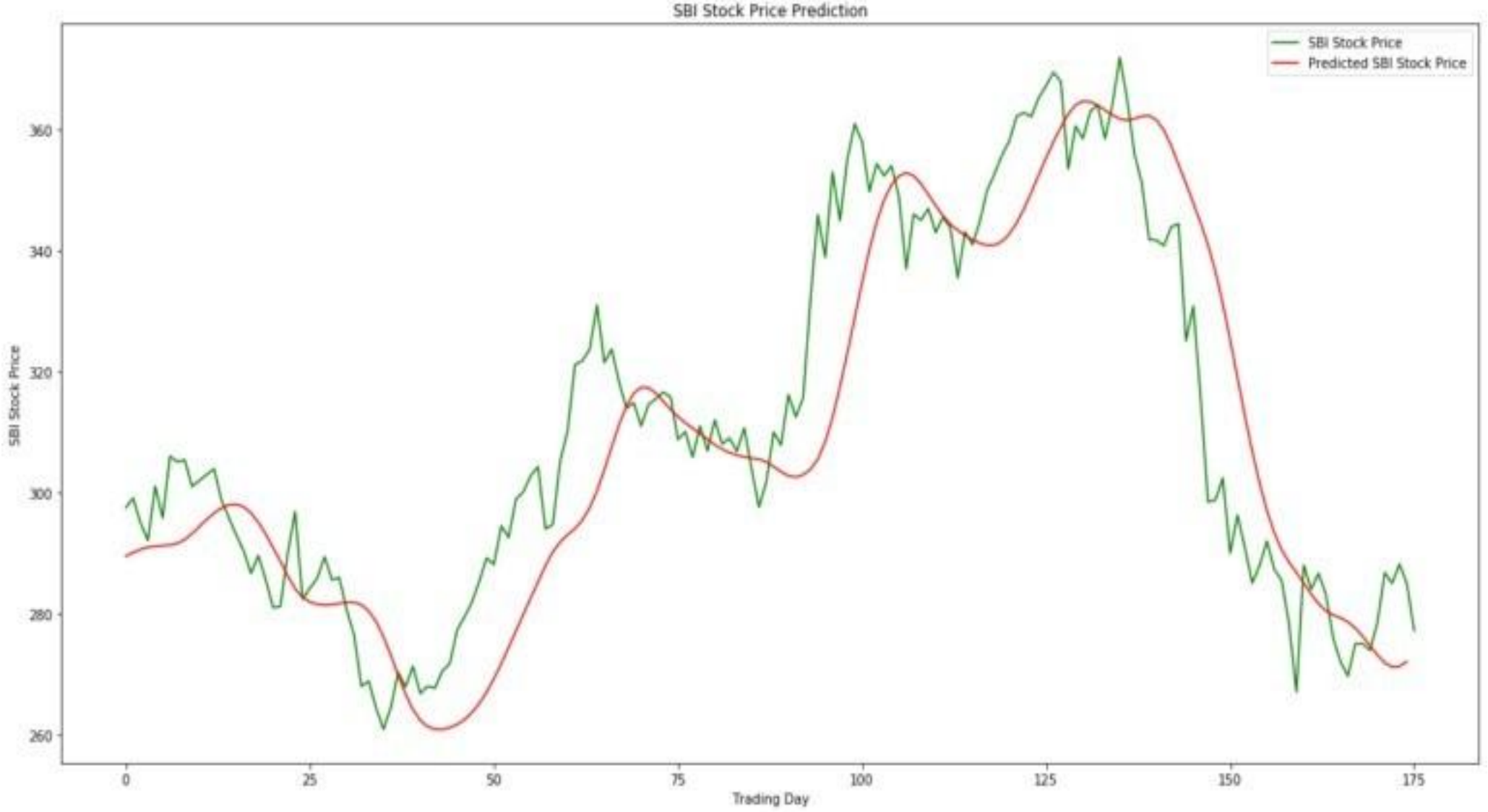
Por que redes neurais recorrentes?

- Elementos aparecem em uma determinada ordem e não são independentes uns dos outros
- A ordem é importante!

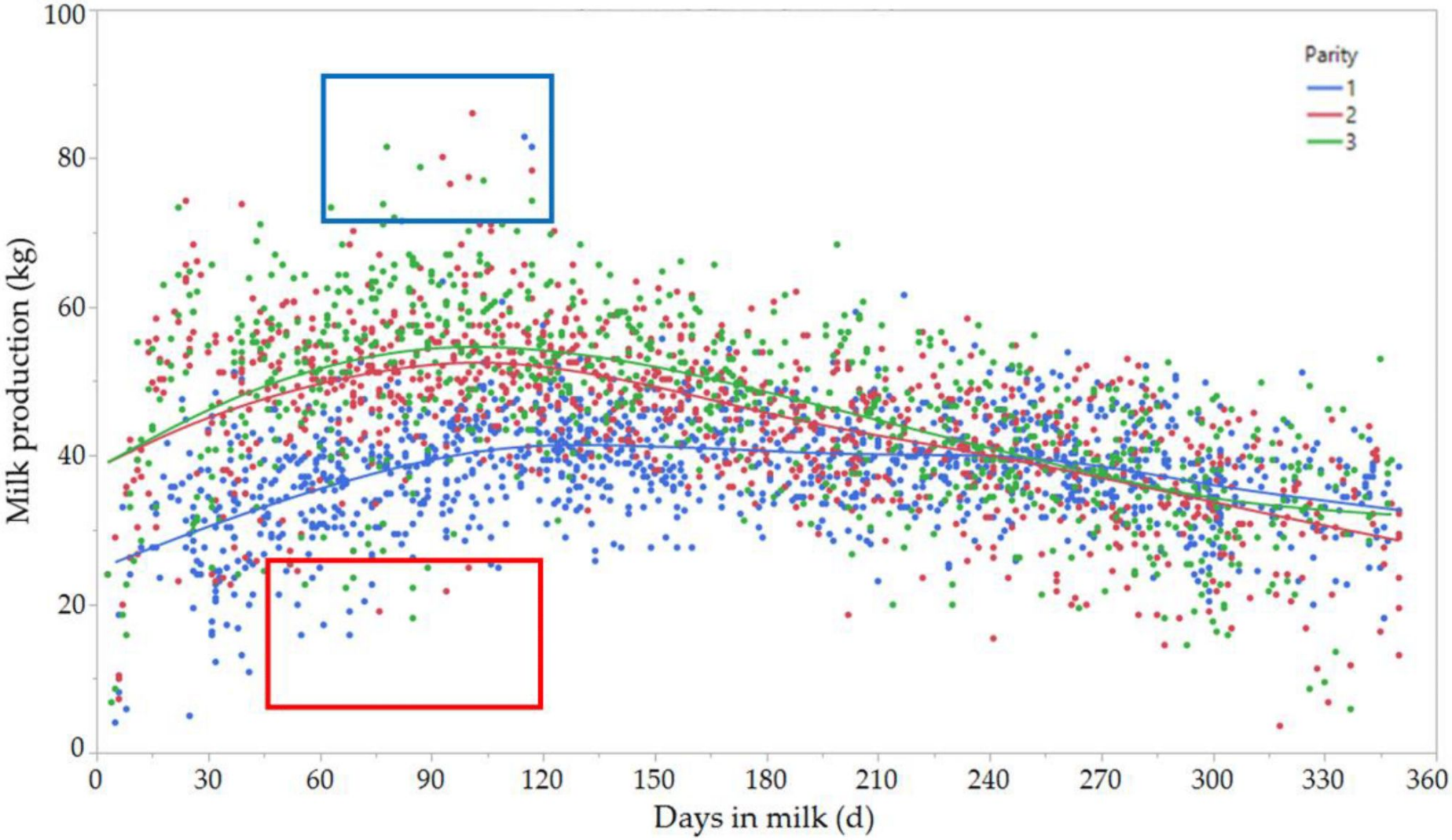
Por que redes neurais recorrentes?



Por que redes neurais recorrentes?



Por que redes neurais recorrentes?



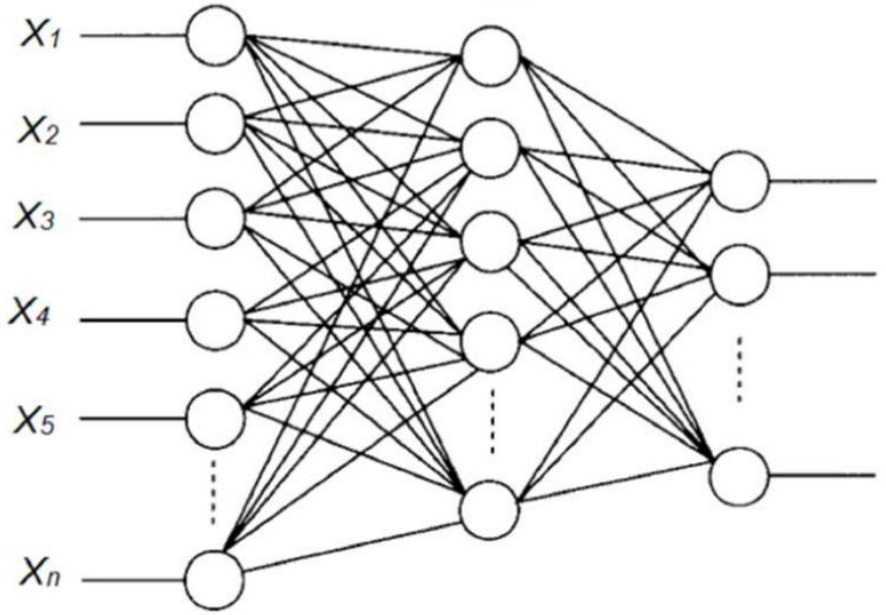
Milk production (kg; vertical axis) by days in milk (DIM; horizontal axis) for different parity groups. The upper blue square highlights peak milk production between 60 and 120 DIM. The lower red rectangle highlights problem cows.
Parity 1 (lactation = 1; blue dots),
Parity 2 (lactation = 2; red dots),
Parity 3 (lactation > 3; green dots).
Continuous lines represent the average milk production for each parity group by days in milk.

Fonte: Monitoring and Improving the Metabolic Health of Dairy Cows during the Transition Period. 2021

Por que redes neurais recorrentes?

"*Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam eget ligula eu lectus lobortis condimentum. Aliquam nonummy auctor massa. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Nulla at risus. Quisque purus magna, auctor et, sagittis ac, posuere eu, lectus. Nam mattis, felis ut adipiscing.*"

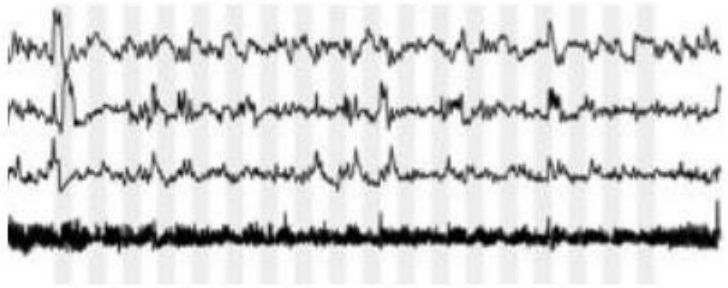
?



Sequências

"This morning I took the dog for a walk."

Texto



Sinais médicos



Áudio

Sequências - Problemas

Ordem

- O jogo foi bom, não foi ruim
- O jogo foi ruim, não foi bom

Sequências - Problemas

Ordem

- O jogo foi bom, não foi ruim
- O jogo foi ruim, não foi bom

Dependência de longo prazo

- Os poucos dias que passei na Espanha foram muito enriquecedores, e tive a oportunidade de aprender a falar_____,

Sequências - Problemas

Ordem

- O jogo foi bom, não foi ruim
- O jogo foi ruim, não foi bom

Dependência de longo prazo

- Os poucos dias que passei na Espanha foram muito enriquecedores, e tive a oportunidade de aprender a falar_____, um idioma muito parecido com o português.

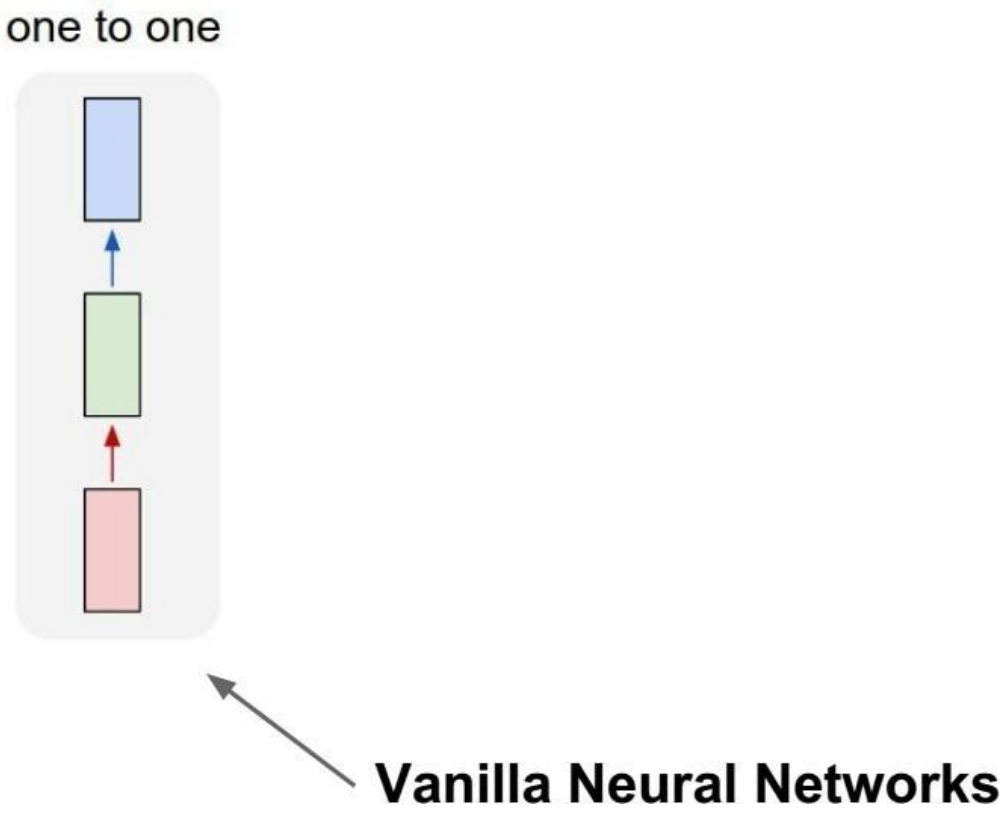
Sequências - Problemas

Ordem
- O jogo foi bom, não foi ruim
- O jogo foi ruim, não foi bom

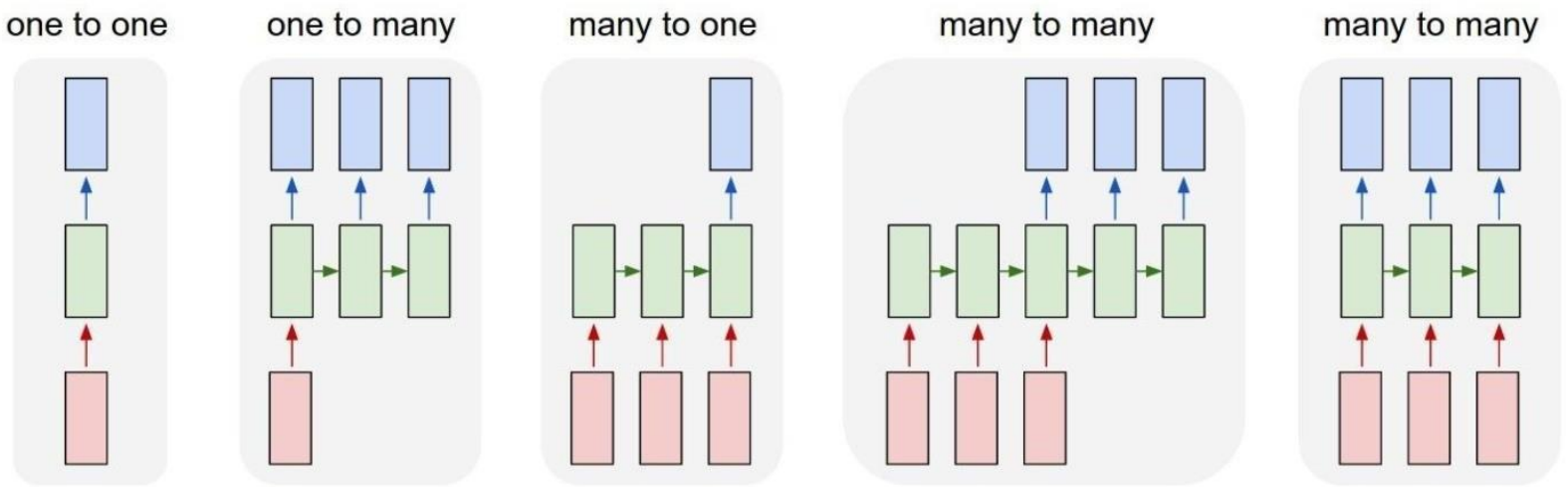
Dependência de longo prazo
- Os poucos dias que passei na espanha foram muito enriquecedores, e tive a oportunidade de aprender a falar_____, um idioma muito parecido com o português.

Compartilhamento de parâmetros (compartilhamento no tempo/ordem)
- Os mesmos parâmetros (pesos) têm que funcionar para todas as sequências…

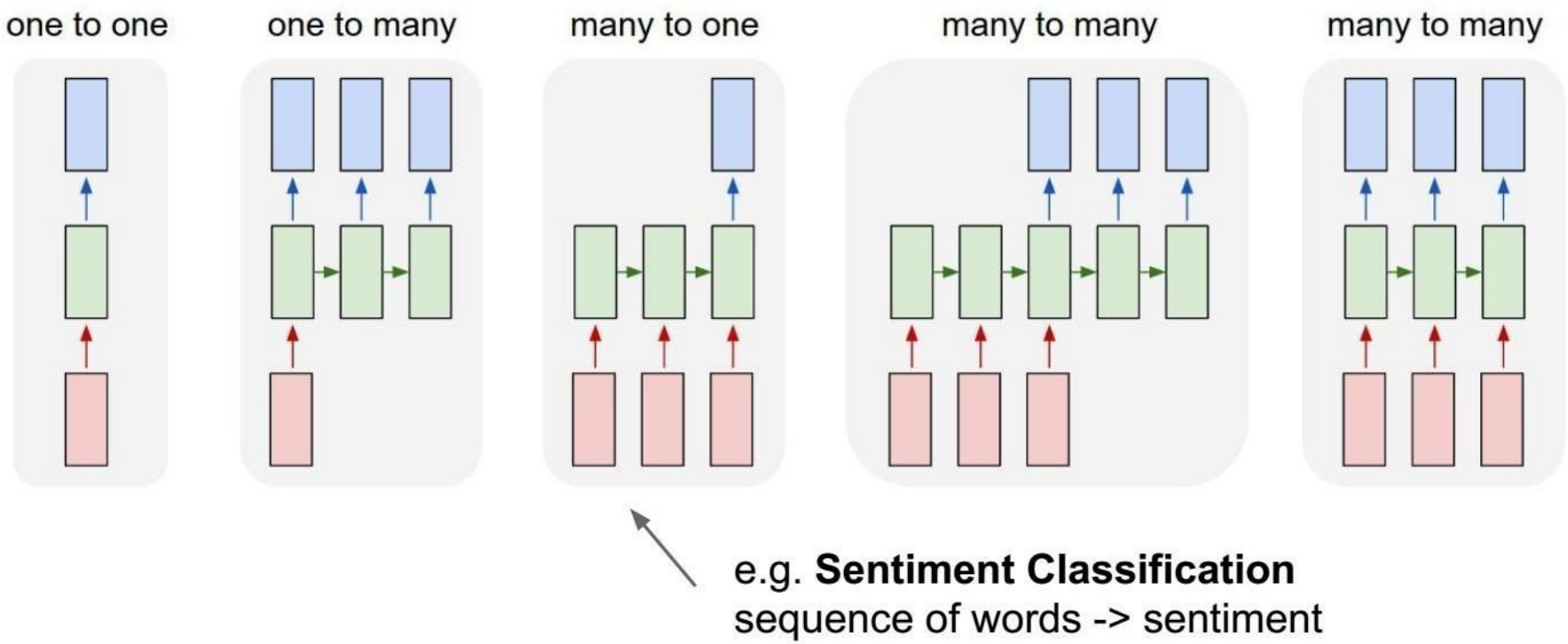
Recurrent Neural Networks



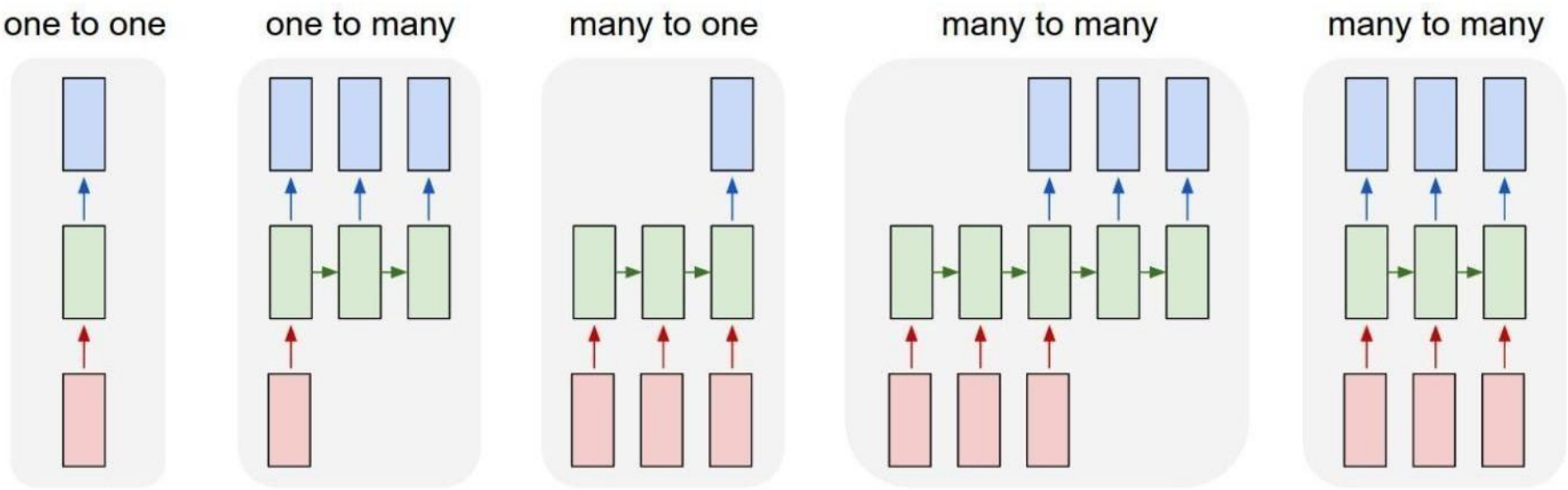
Recurrent Neural Networks



Recurrent Neural Networks



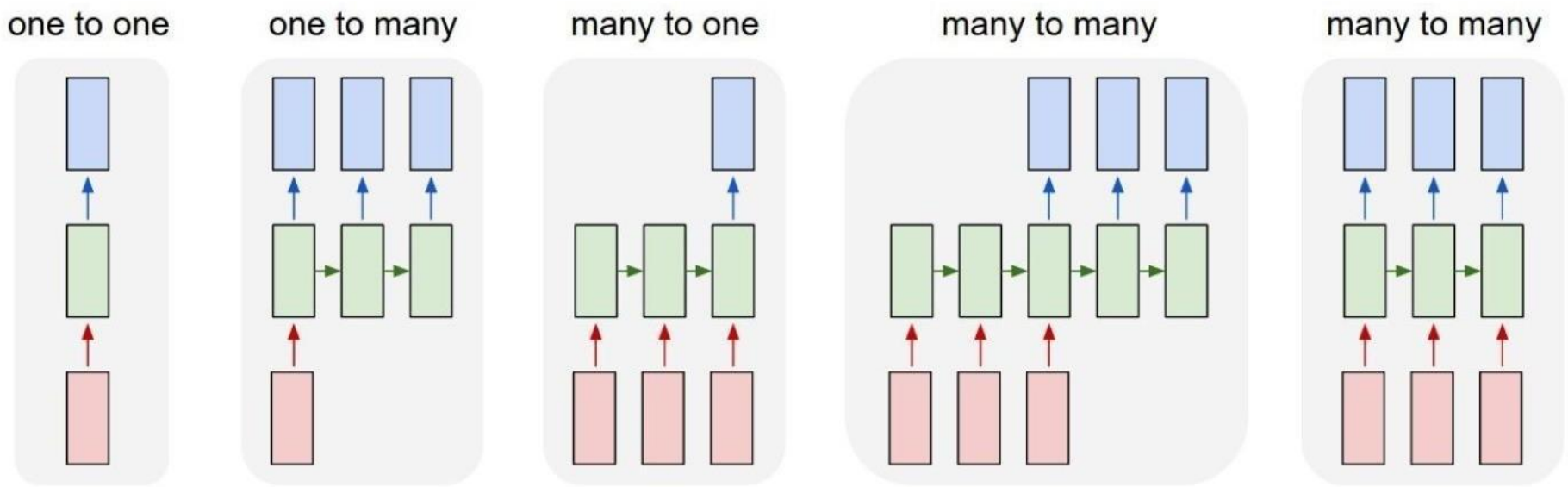
Recurrent Neural Networks



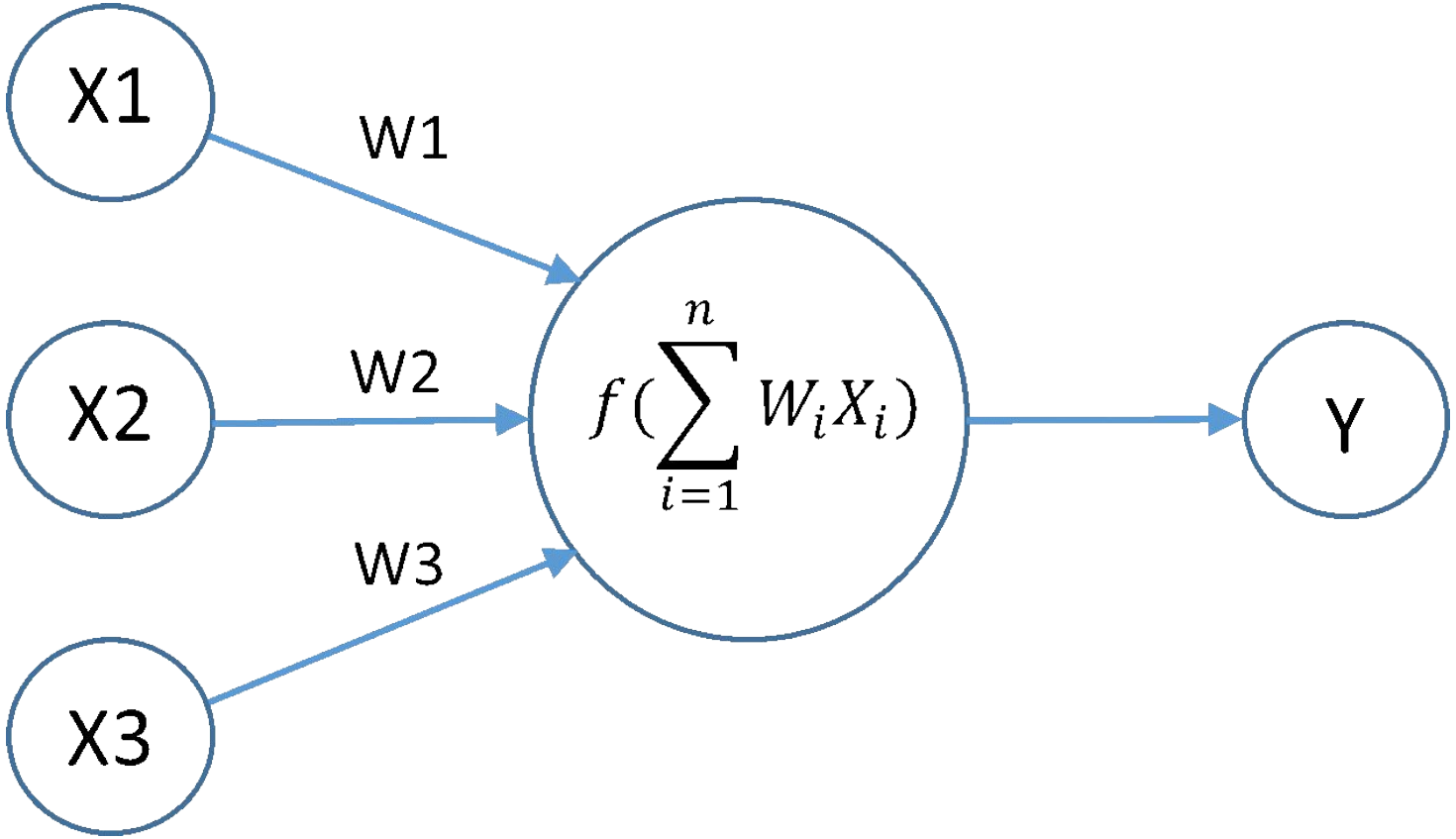
e.g. **Machine Translation**
seq of words -> seq of words

… precisa ler todo o texto para depois fazer a tradução

Recurrent Neural Networks

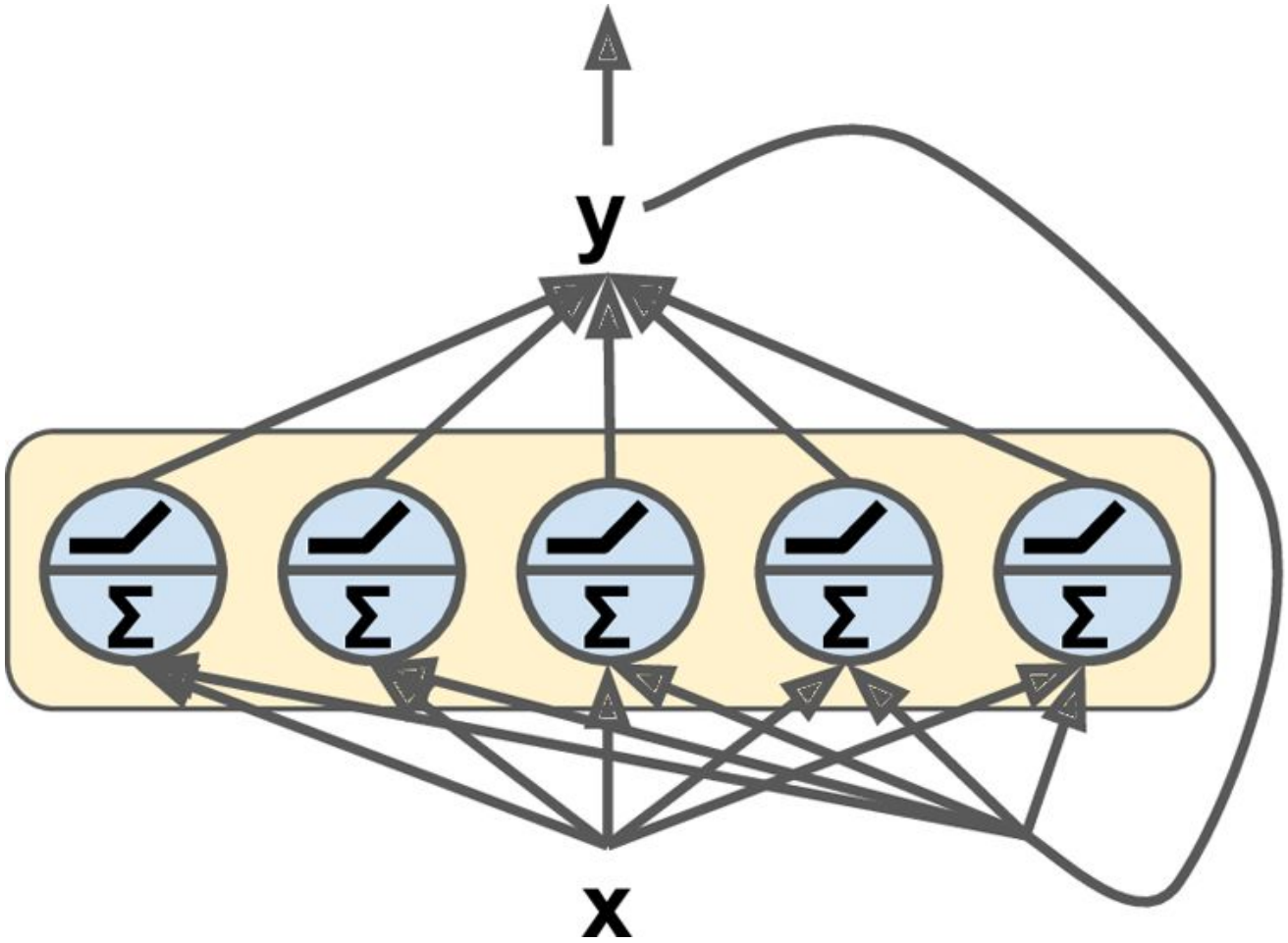


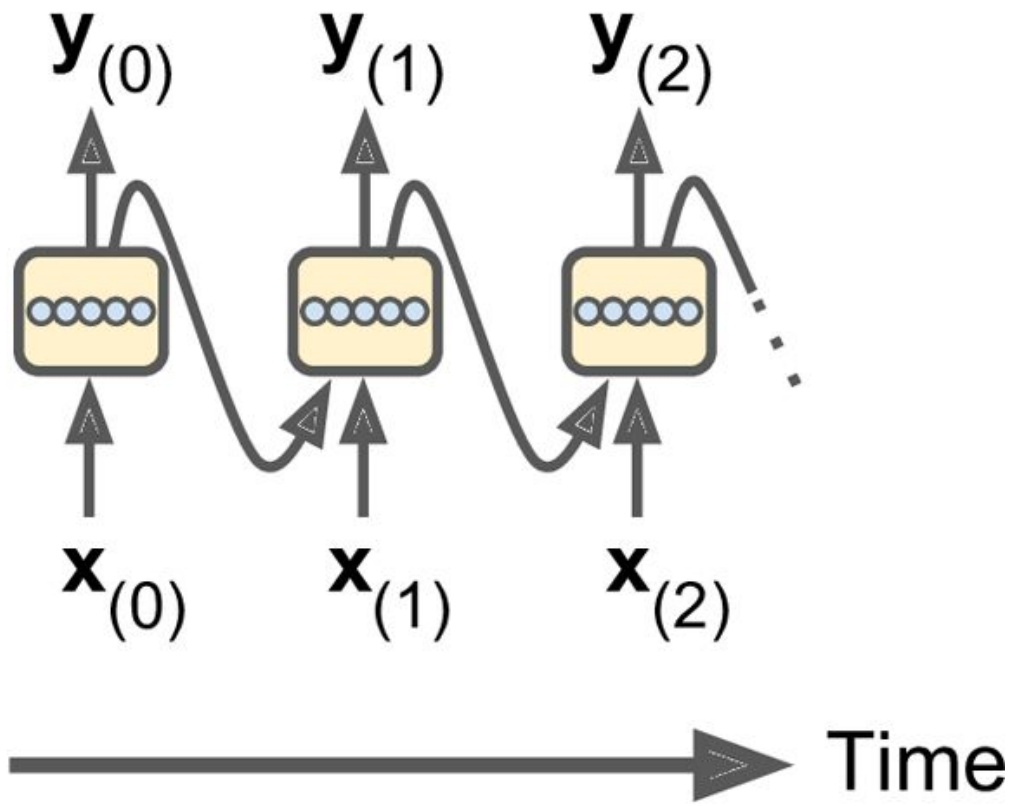
Vanilla Neural Network



mas preciso processar mútiplas entradas n

RNN





RNN

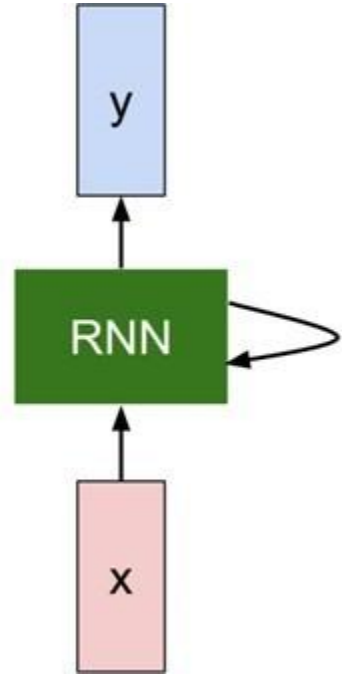
$$h_t = f_W(h_{t-1}, x_t)$$

Entrada no tempo t

Novo estado

Estado anterior

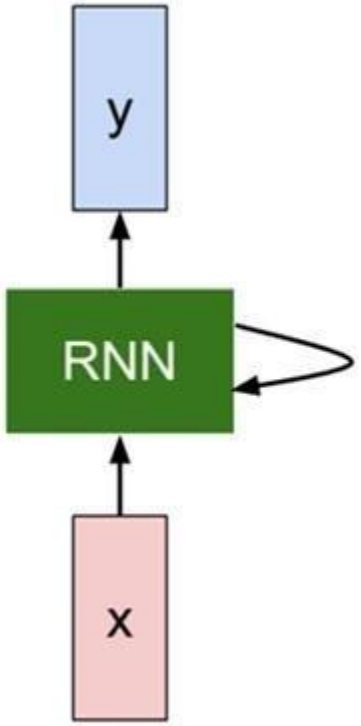
Alguma função de ativação com parâmetros W



conceito recorrente dentro do neurônio...

O neurônio passa a ter um estado h - "memória" interna do neurônio!
... a função de ativação atua no input do tempo t e no estado do tempo t-1

RNN



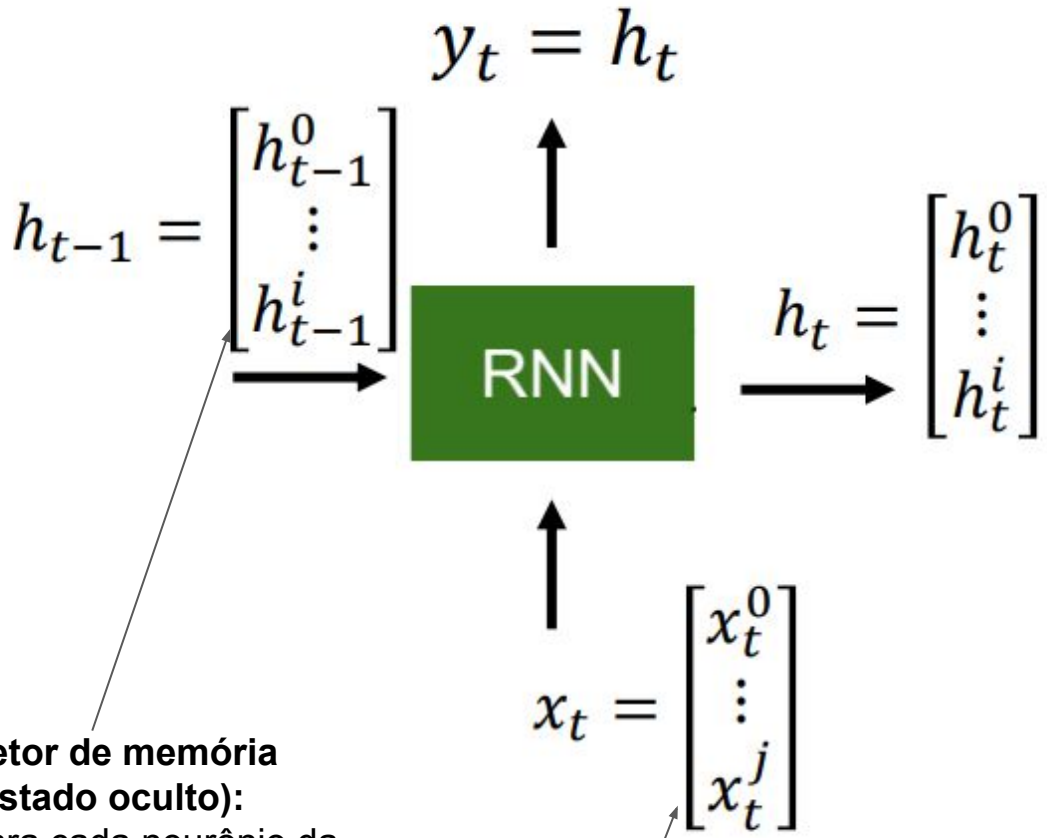
$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

matriz de pesos do estado interno

matriz para operar os inputs (matriz de pesos originais da rede)

RNN



Vetor de memória (estado oculto): para cada neurônio da minha rede, tenho um elemento de memória associado.

Vetor de entrada: j dimensões

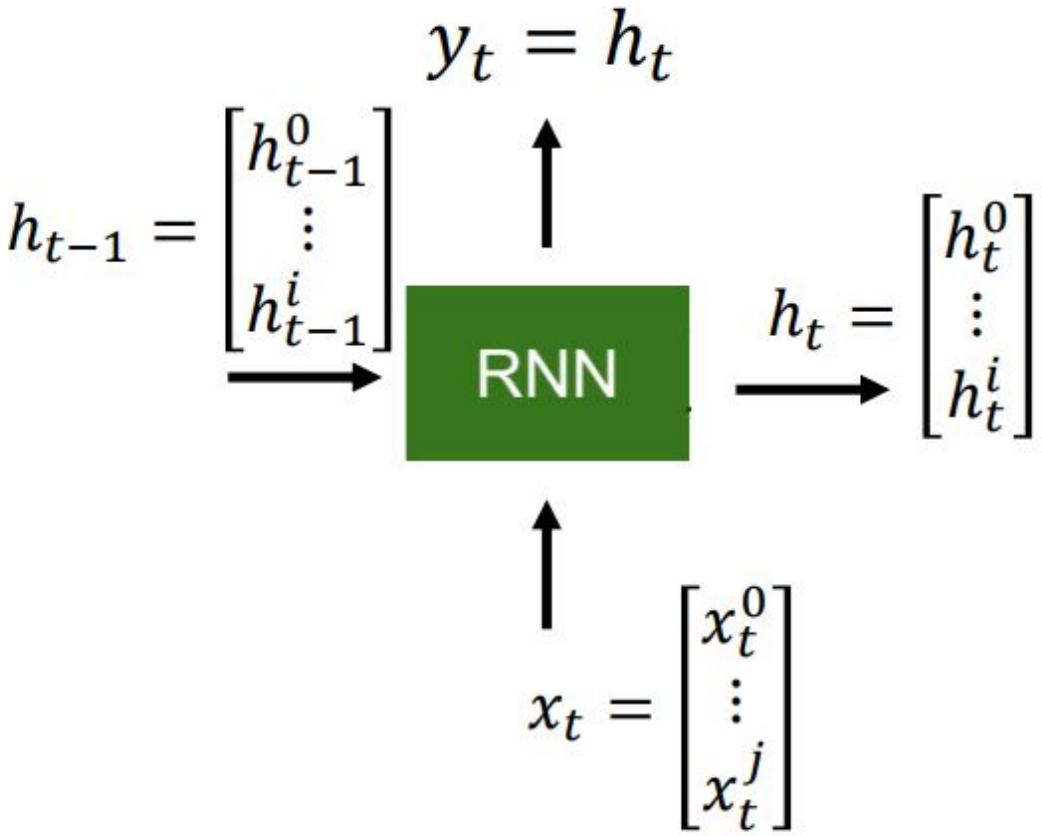
$$h_t = \tanh(W_{hh} h_{t-1} + W_{hx} x_t)$$

$$W_{hh} = \begin{bmatrix} w_{00} & \cdots & w_{0i} \\ \vdots & \ddots & \vdots \\ w_{i0} & \cdots & w_{ii} \end{bmatrix} = w_{ij} \in \mathbb{R}^{i \times i}$$

$$W_{hx} = \begin{bmatrix} w_{00} & \cdots & w_{0j} \\ \vdots & \ddots & \vdots \\ w_{i0} & \cdots & w_{ij} \end{bmatrix} = w_{ij} \in \mathbb{R}^{i \times j}$$

$$h_t = \tanh([W_{hh} \quad W_{hx}] \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

RNN



$$h_t = \tanh(W_{hh} h_{t-1} + W_{hx} x_t)$$

matriz de pesos quadrada sobre os i neurônios da rede

$$W_{hh} = \begin{bmatrix} w_{00} & \cdots & w_{0i} \\ \vdots & \ddots & \vdots \\ w_{i0} & \cdots & w_{ii} \end{bmatrix} = w_{ij} \in \mathbb{R}^{i \times i}$$

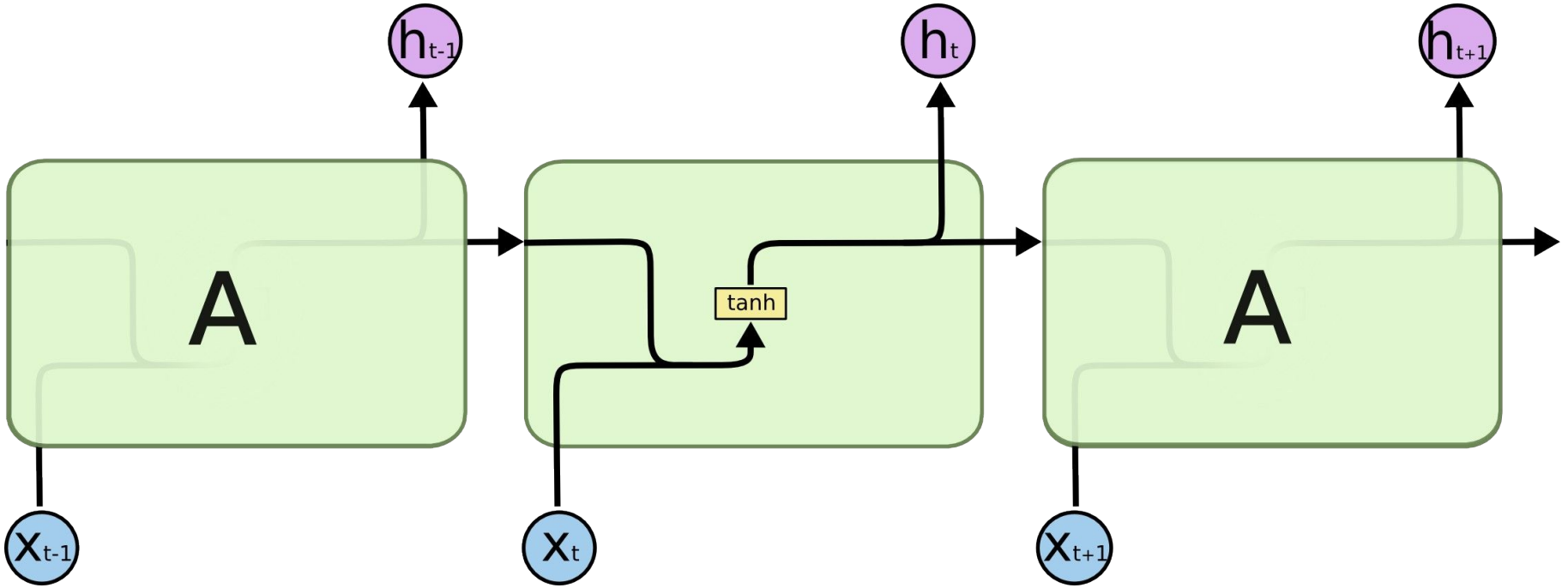
matriz de pesos sobre entrada por neurônios da rede (a pesos para saída do tempo t como entrada de t+1)

$$W_{hx} = \begin{bmatrix} w_{00} & \cdots & w_{0j} \\ \vdots & \ddots & \vdots \\ w_{i0} & \cdots & w_{ij} \end{bmatrix} = w_{ij} \in \mathbb{R}^{i \times j}$$

$$h_t = \tanh([W_{hh} \quad W_{hx}] \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

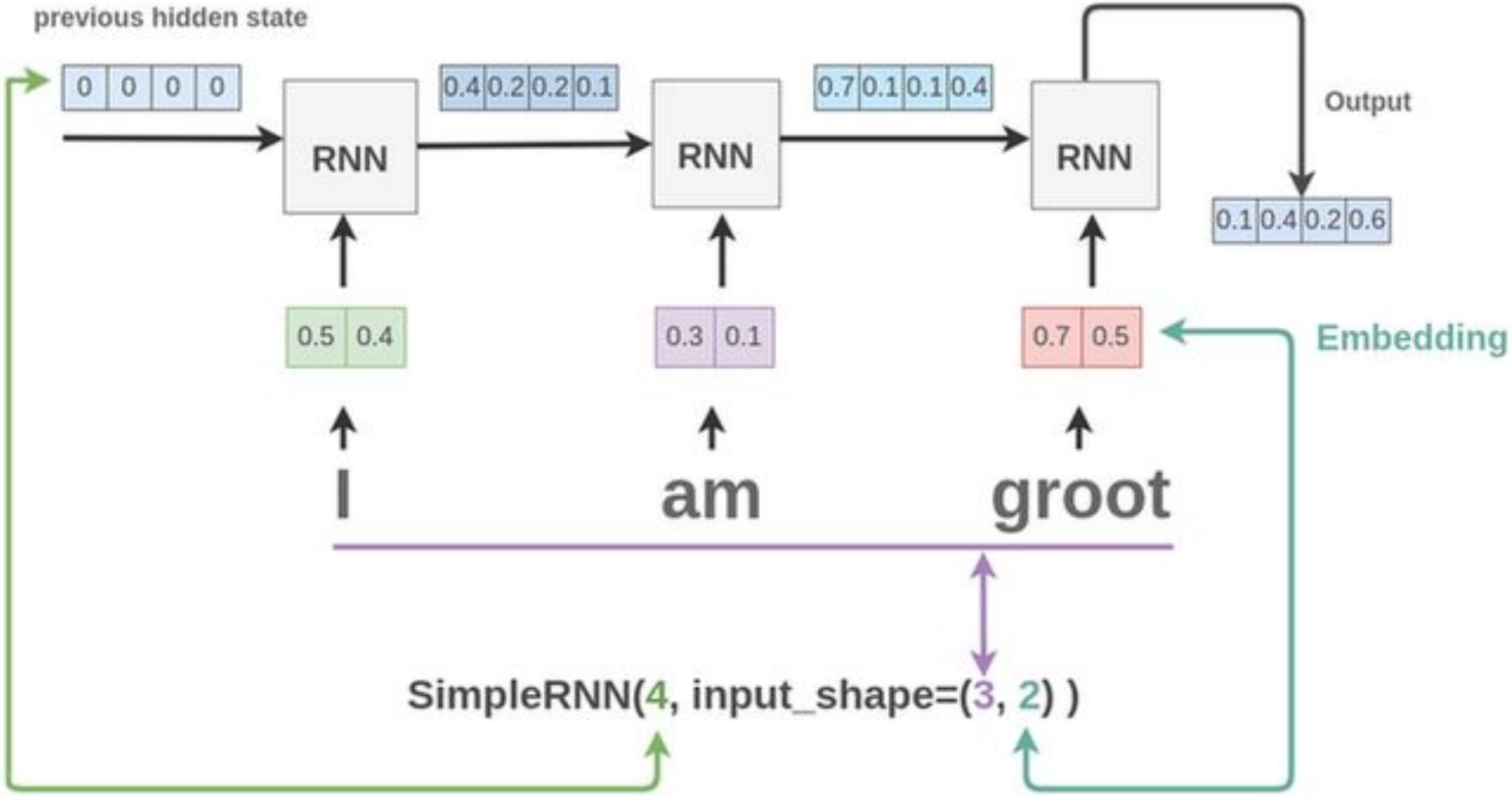
concateno "a compreensão do estado atual" com

RNN



fonte:
http://colah.github.io/posts/2015-08-Understanding-LSTMs/

RNN



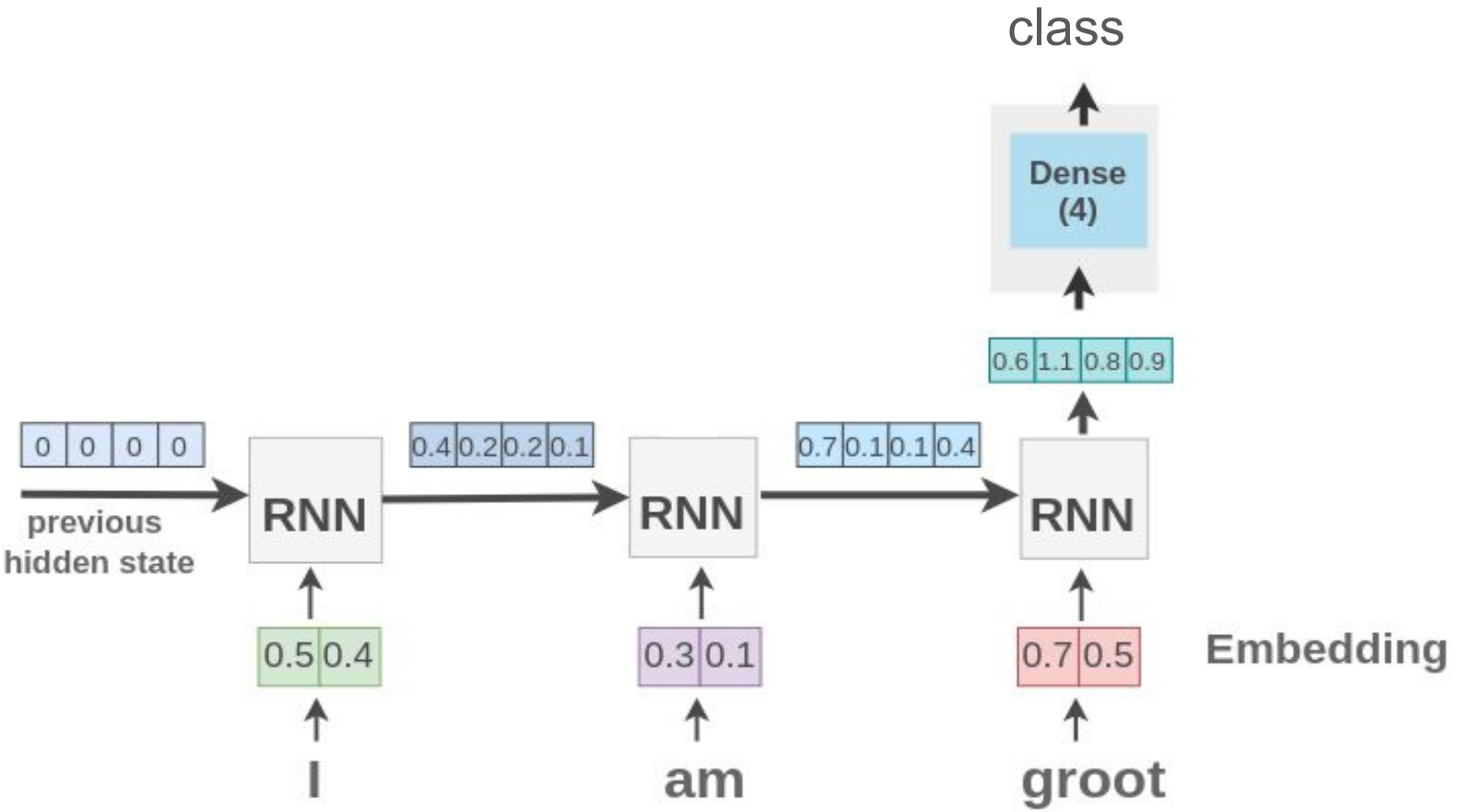

```
Código:
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2)))
```

fonte: https://amitness.com/2020/04/recurrent-layers-keras/

RNN - Classificação

Many to one…

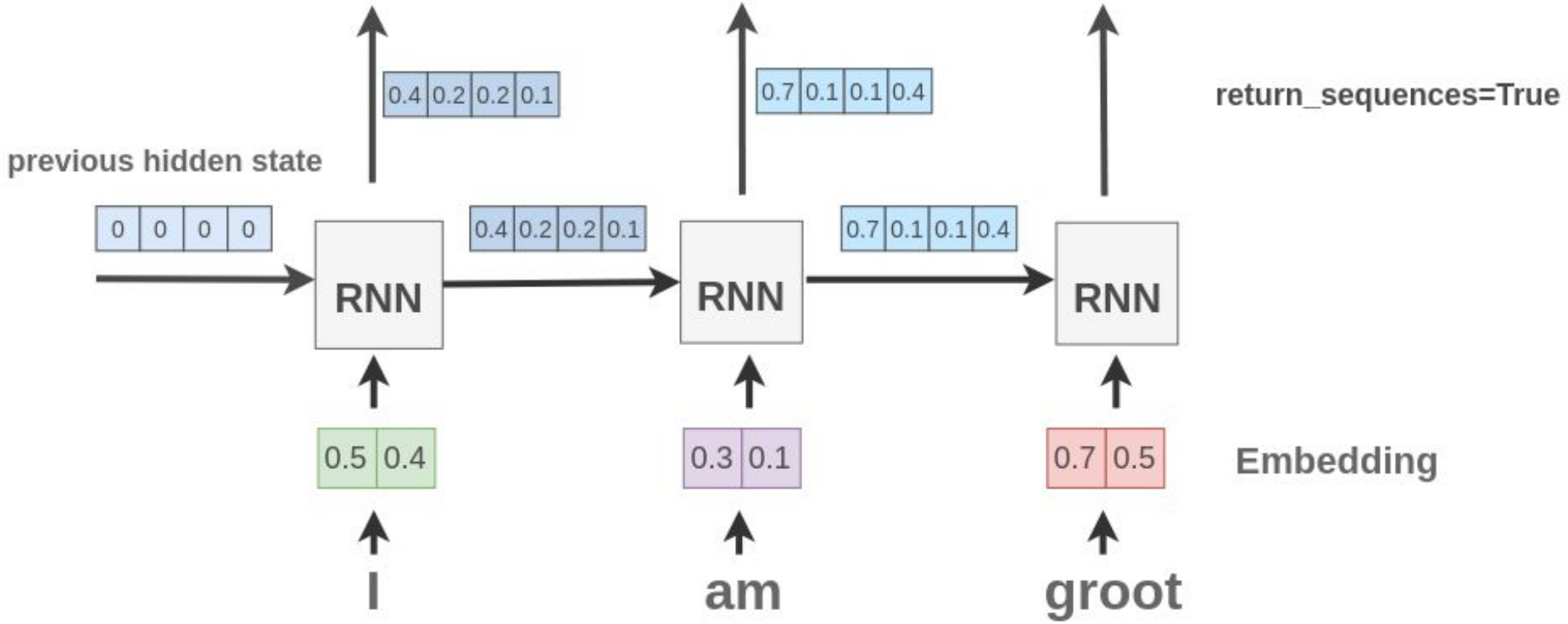
ex. de aplicação: queremos saber qual a classe da sentença (ótima, boa, ruim ou péssima).




```
Código:
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2)))
model.add(Dense(4, activation='softmax')
```

fonte: https://amitness.com/2020/04/recurrent-layers-keras/

RNN

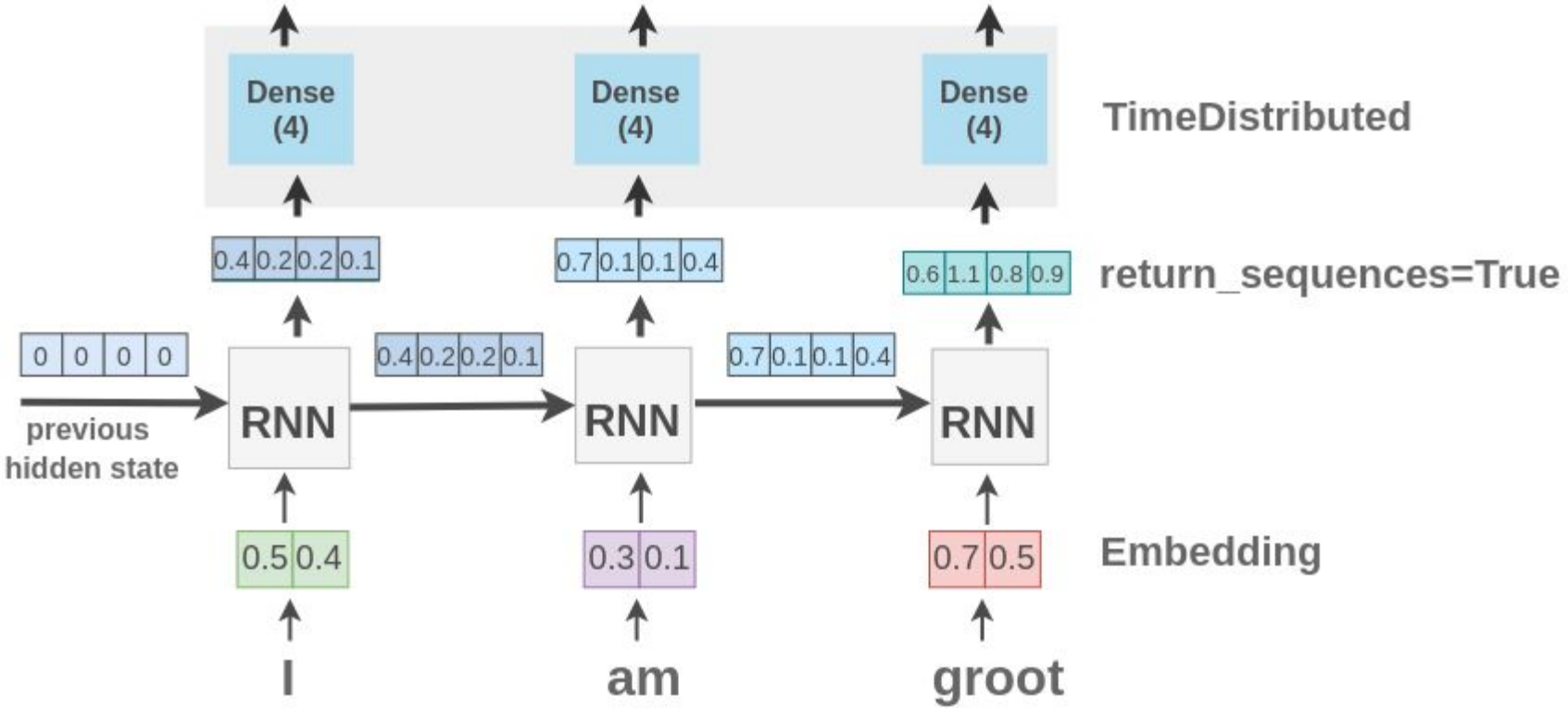



```
Código:
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2), return_sequences=True))
```

fonte: https://amitness.com/2020/04/recurrent-layers-keras/

RNN - Classificação

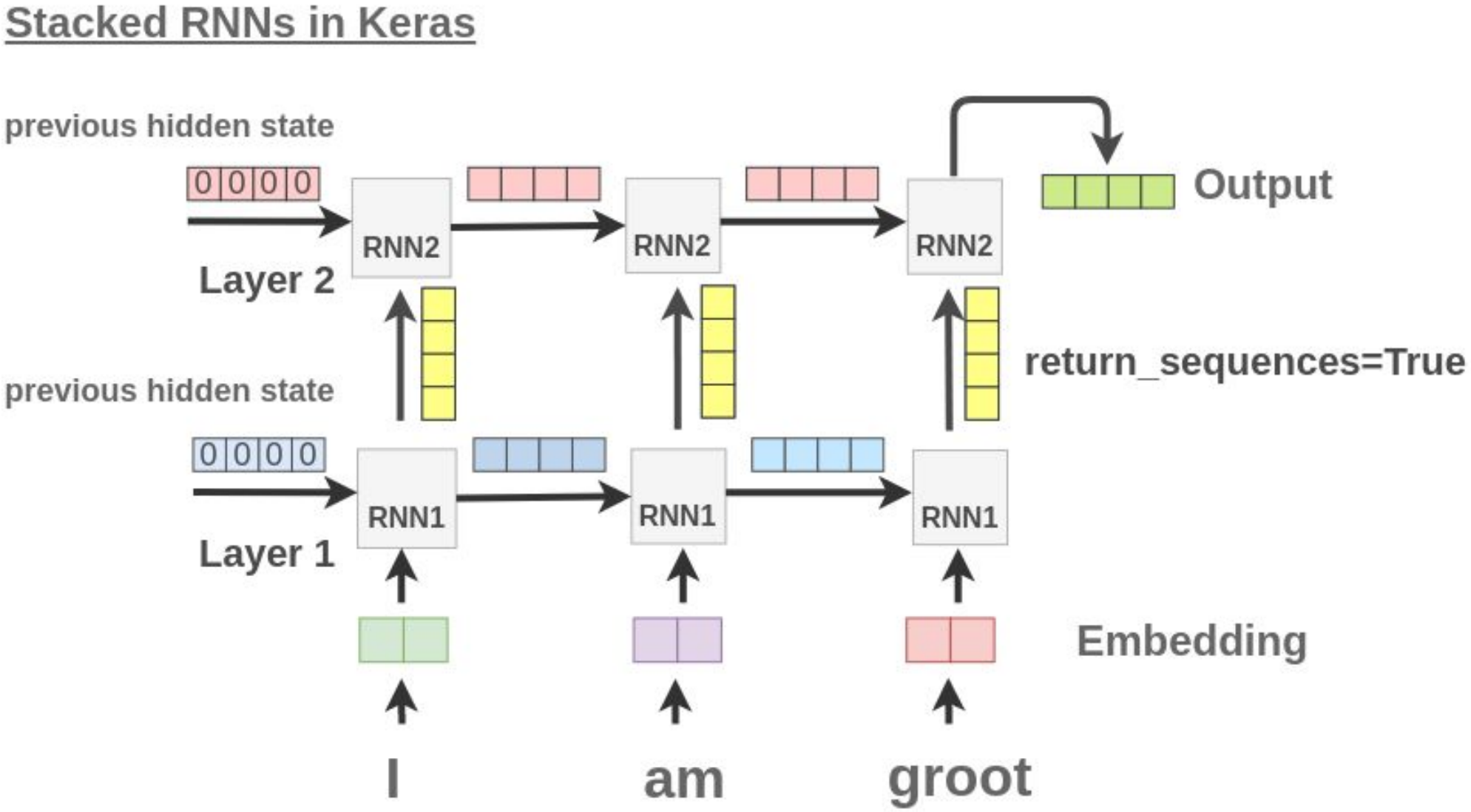
Many to Many…
ex. de aplicação: queremos saber qual a classe da palavra (verbo, adjetivo, substantivo ou outros)




```
Código:
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2), return_sequences=True))
model.add(TimeDistributed(Dense(4, activation='softmax')))
```

fonte: https://amitness.com/2020/04/recurrent-layers-keras/

RNN




```
Código:
model = Sequential()
model.add(SimpleRNN(4, input_shape=(3, 2), return_sequences=True))
model.add(SimpleRNN(4))
```

fonte: https://amitness.com/2020/04/recurrent-layers-keras/

ex.: Usando NLP Generativa com RNNs

- Seja o problema de prever qual a próxima palavra que quero gerar (para construir um texto com RNN)

- Podemos colocar um **softmax** na saída da RNN mapeando cada possível palavra (i.e., cada palavra é uma classe):

$$y_t = softmax(W_{hy} h_t)$$

- Teoricamente, poderíamos estimar perfeitamente a palavra que tem maior probabilidade de ocorrer na sequência (afinal, redes neurais são aproximadores universais):

$$P(x_{t+1} | x_t, \ldots, x_0) = y_t$$

PROBLEMA com RNN:

Entretanto…

Só falta saber os pesos da rede.

Como vamos aplicar o Backpropagation na RNN?

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W}$$

Derivada parcial do erro em relação aos pesos da RNN

$$\frac{\partial E_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W}$$

Derivada do erro em relação à saída da rede

Derivada da saída da rede em relação ao estado t

Derivada do estado t em relação aos k estados anteriores

Derivada dos estados anteriores em relação à matriz de pesos

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W}$$

$$\frac{\partial E_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \boxed{\frac{\partial h_t}{\partial h_k}} \frac{\partial h_k}{\partial W}$$

Produtório muito grande de derivadas parciais!

RNN

Problemas:

- **Explosão/sumiço dos Gradientes!**
  Inclusão de ruído devido ao Vanishing/Exploding gradiente
  - Clipping pode ajudar no exploding
    - `ex.: SGD(lr=0.01, clipvalue=1.0)`

- Dificuldade em lidar com **dependências de longo prazo**
  - João entrou na sala. José também. Já é tarde e ambos estão atrasados. João disse oi para____.
    - difícil de "lembrar" coisas antigas

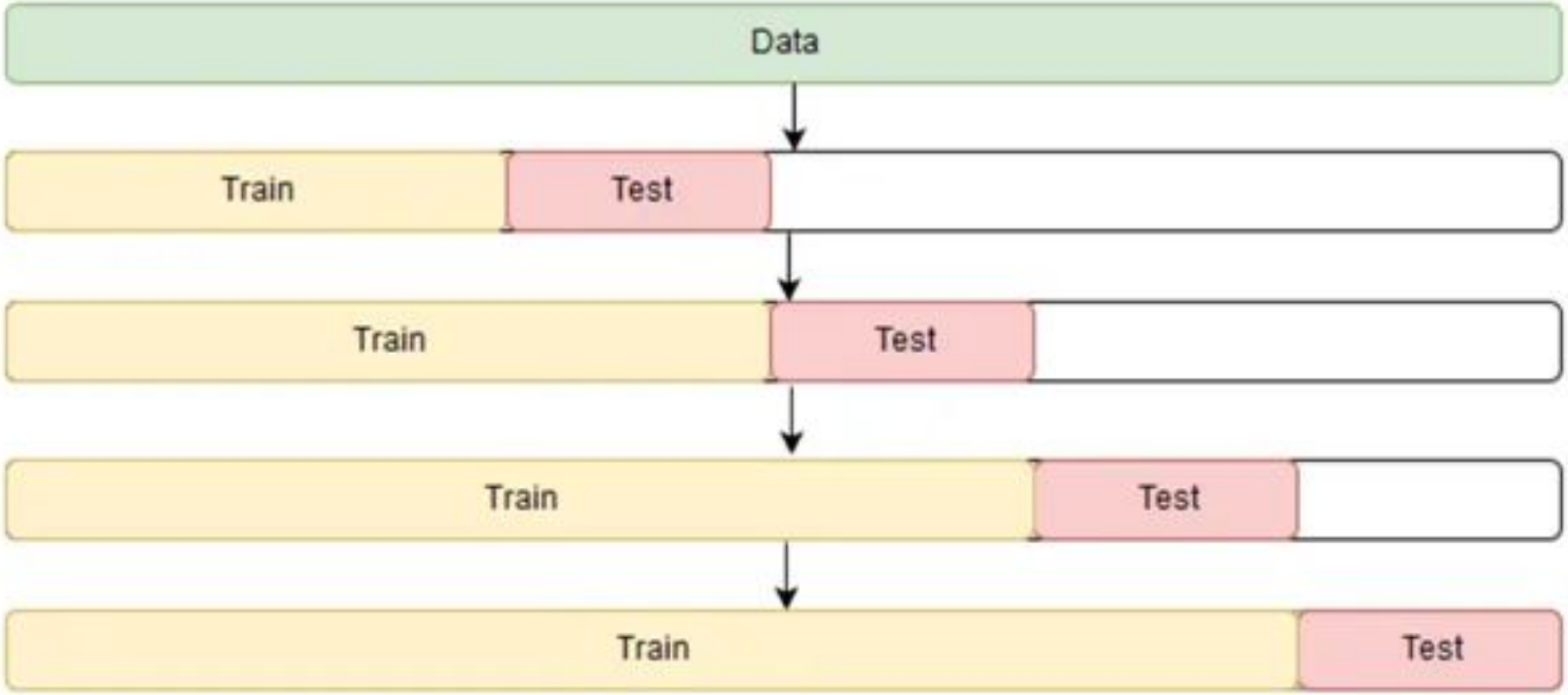
- Dificuldade em lidar com **ruído**
  - ruído e informações irrelevantes não são esquecidas
    - "tudo" é armazenado na memória

Teoricamente funciona, mas na prática é difícil treinar para problemas complexos

Validação Cruzada em Séries Temporais

Caso1: Uma única série temporal

- Cuidado! não podemos assumir que temos informação privilegiada de dados futuros

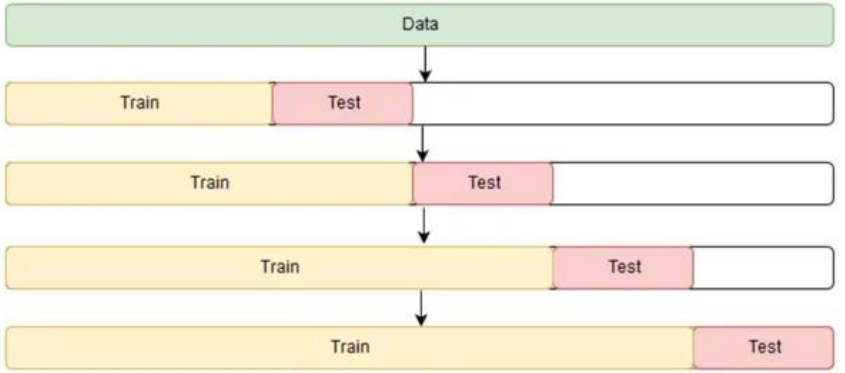


Validação Cruzada em Séries Temporais

Caso 1: Uma única série temporal

- Cuidado! não podemos assumir que temos informação privilegiada de dados futuros

```
import numpy as np
from sklearn.model selection import TimeSeriesSplit
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4], [1, 2], [3, 4]])
y = np.array([1, 2, 3, 4, 5, 6])
tscv = TimeSeriesSplit()
print(tscv)
TimeSeriesSplit(max train size=None, n splits=3)
for train index, test index in tscv.split(X):
    print("TRAIN:", train index, "TEST:", test index)
    X train, X test = X[train index], X[test index]
    y_train, y_test = y[train_index], y[test_index]
```




```
TimeSeriesSplit(max_train_size=None, n_splits=3)
TRAIN: [0 1 2] TEST: [3]
TRAIN: [0 1 2 3] TEST: [4]
TRAIN: [0 1 2 3 4] TEST: [5]
```

Validação Cruzada em Séries Temporais

Caso 2: Múltiplas séries temporais independentes e identicamente distribuídas.

- Nesse caso, podemos aplicar a validação cruzada tradicional
  - Nesse caso, podemos dividir a base em séries de treinamento e séries de teste.

Ex.:
Série 1 = [1, 2, 45,5 ,6, …]
Série 2 = [51, 12, 4,5 ,56, …]
Série 3 = [1, 2, 45,5 ,36, …]
Série 4 = [31, 2, 45,5 ,6, …]
Série 5 = [21, 2, 45,5 ,76, …]

- Série 1 para teste, série 2 para validação e séries 3 e 4 para treinamento.
- Série 2 para teste, série 3 para validação e séries 1 e 4 para treinamento.
- …

Como a RNN "enxerga" instâncias na série?

Cada instância é associada a uma janela da série, com tamanho *j*.

Ex.:
Série 1 = [1, 2, 3, 4, 5, 6, 7, 8, 9…]

Assumindo j=3, para o problema de prever o próximo número da série, temos as seguintes instâncias:

- instância 1
  - x1 = [1, 2, 3], y1=4
- instância 2
  - x2=[2, 3, 4], y2=5
- instância 3
  - x3=[3, 4, 5], y3=6
- ...

RNN

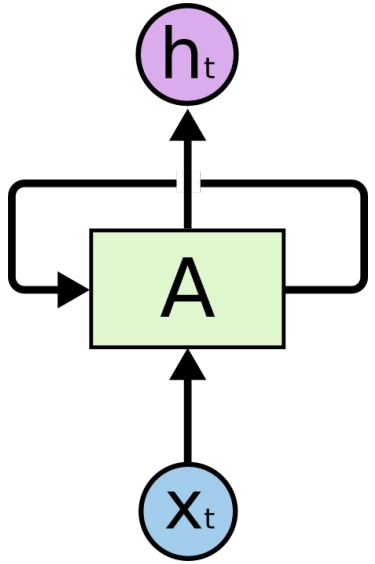
Código:
https://colab.research.google.com/drive/1jbty900Tc-oGu_zTX8FU92rGFOhhNP2T?usp=sharing

Redes Recorrentes...

- Inspiração:
  - Nós humanos não reiniciamos nossos pensamentos do zero a cada segundo!
  - Cada palavra dessa leitura -> entendimento das palavras anteriores
  - Pensamentos têm “persistência”.
  - No aprendizado de máquina tradicional:
    - Não fica claro/explícito como informações prévias podem ser modeladas para auxiliar informações futuras!
  - Redes neurais recorrentes tratam essa questão:
    - Loops internos.
    - persistência de informações a cada etapa.

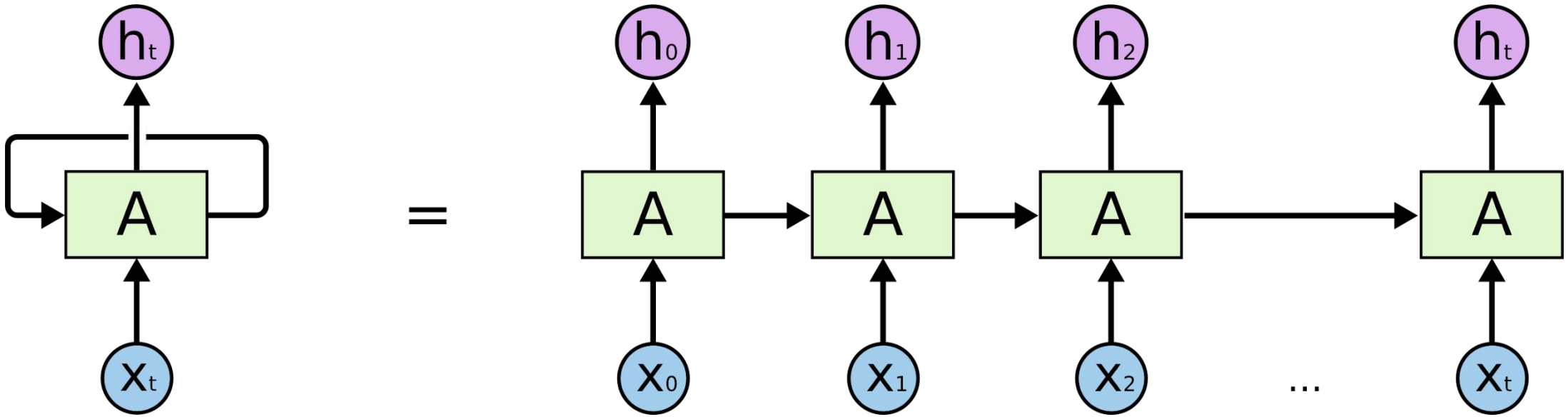
Redes Recorrentes...

- Uma rede A recebe a entrada x_t e dá como saída a resposta h_t.
- O loop interno permite a passagem de informação a cada passo do processamento.



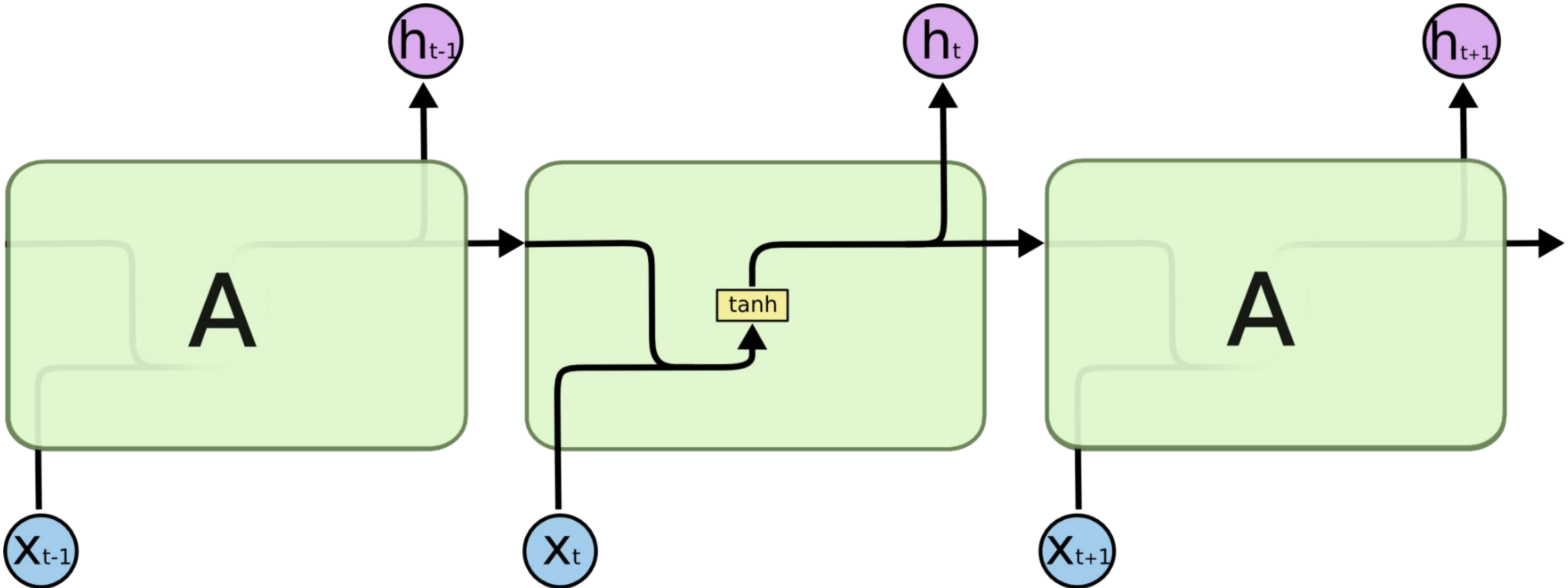
Redes Recorrentes...

- Redes recorrentes podem ser pensadas como múltiplas cópias da mesma rede.
- A cada passada, a versão anterior passa uma “mensagem” para próxima rede.



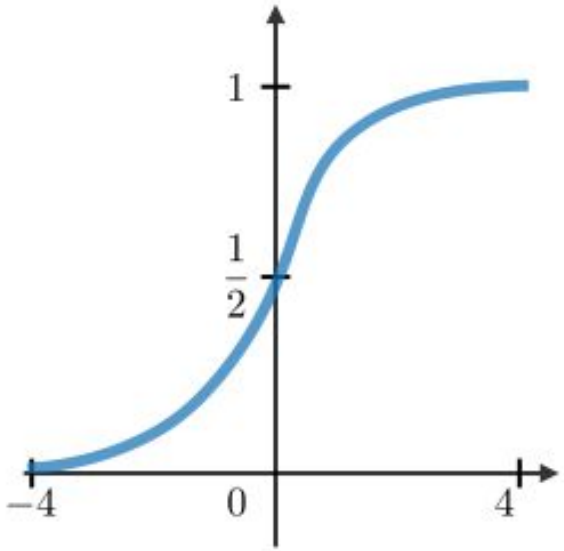
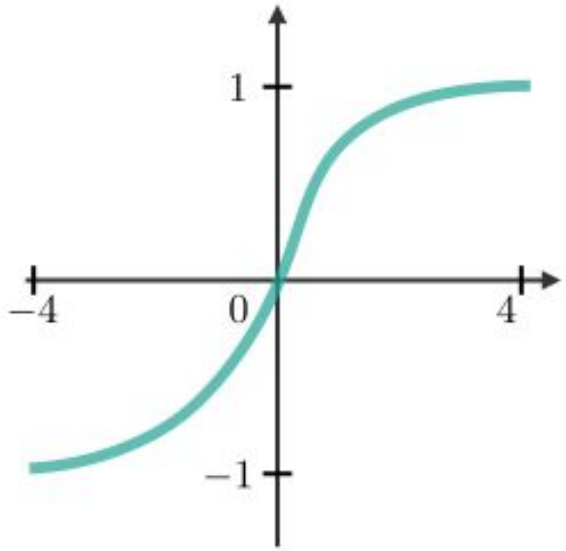
Redes Recorrentes...

- Redes recorrentes podem ser pensadas como múltiplas cópias da mesma rede.
- A cada passada, a versão anterior passa uma “mensagem” para próxima rede.

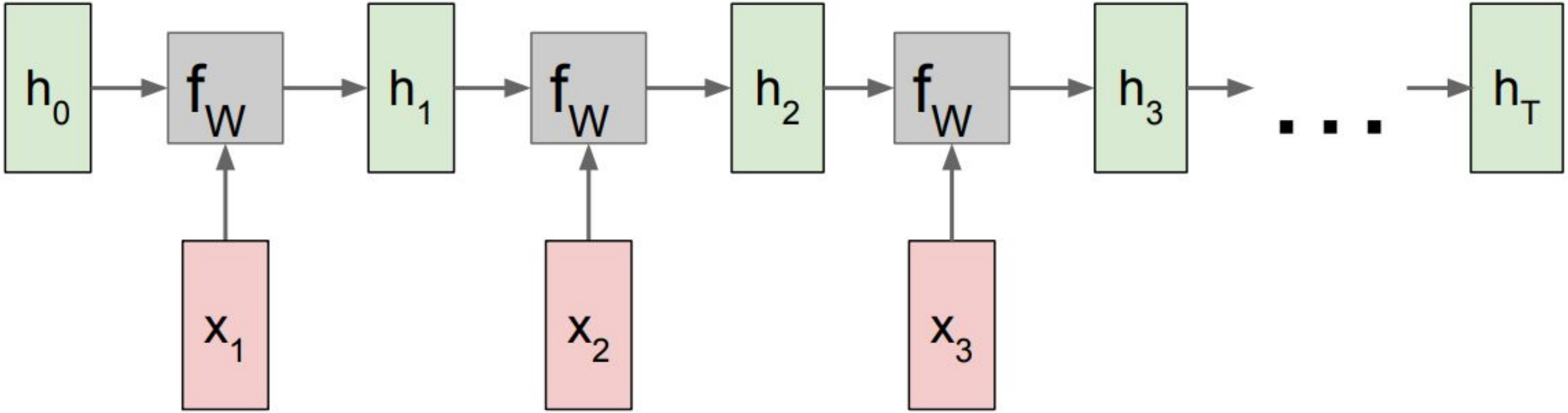


Redes Recorrentes...

- Funções de ativação comuns em redes recorrentes:

Sigmoid	Tanh
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$
	

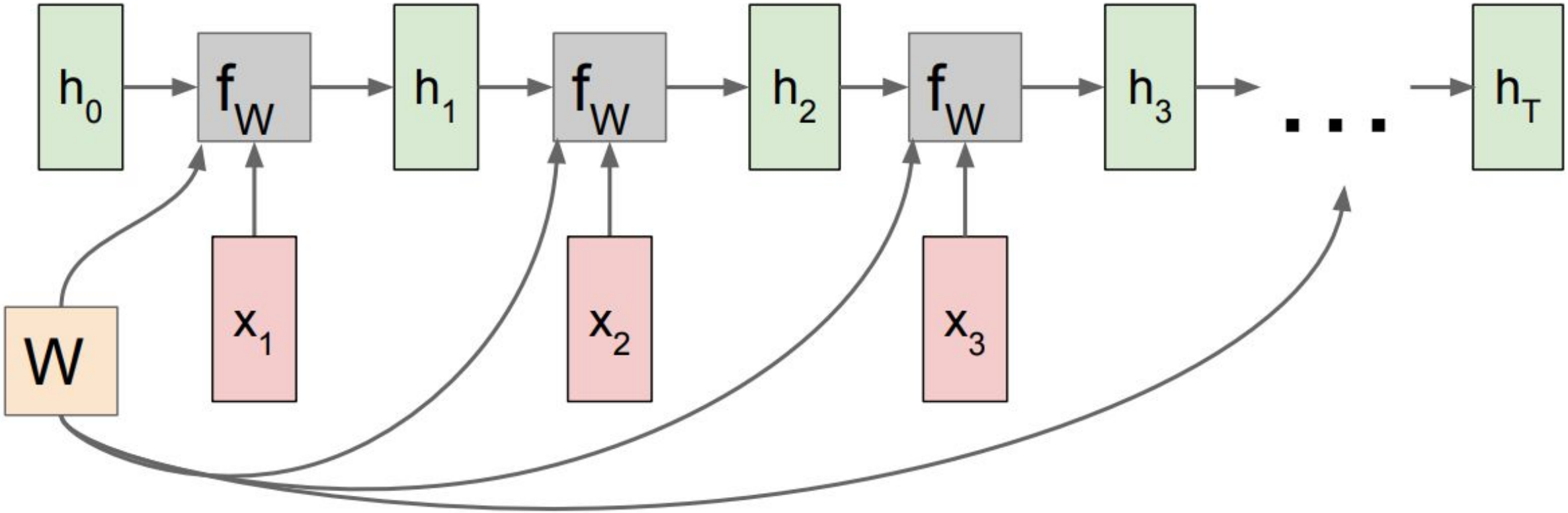
Redes Recorrentes (grafo de computação)...



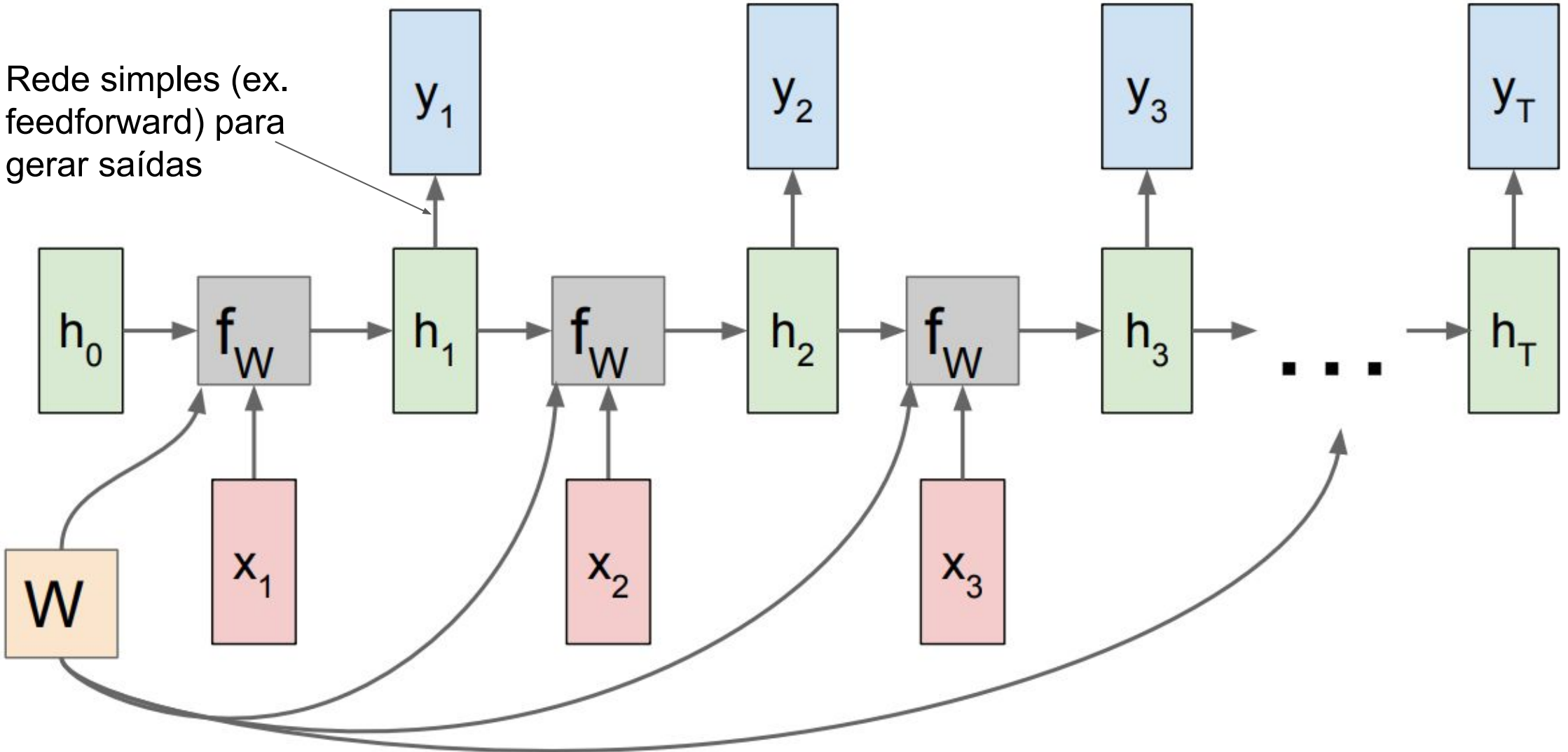
http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture10.pdf

Redes Recorrentes (grafo de computação)...

- Reutiliza a mesma matriz de pesos a cada passo de tempo.

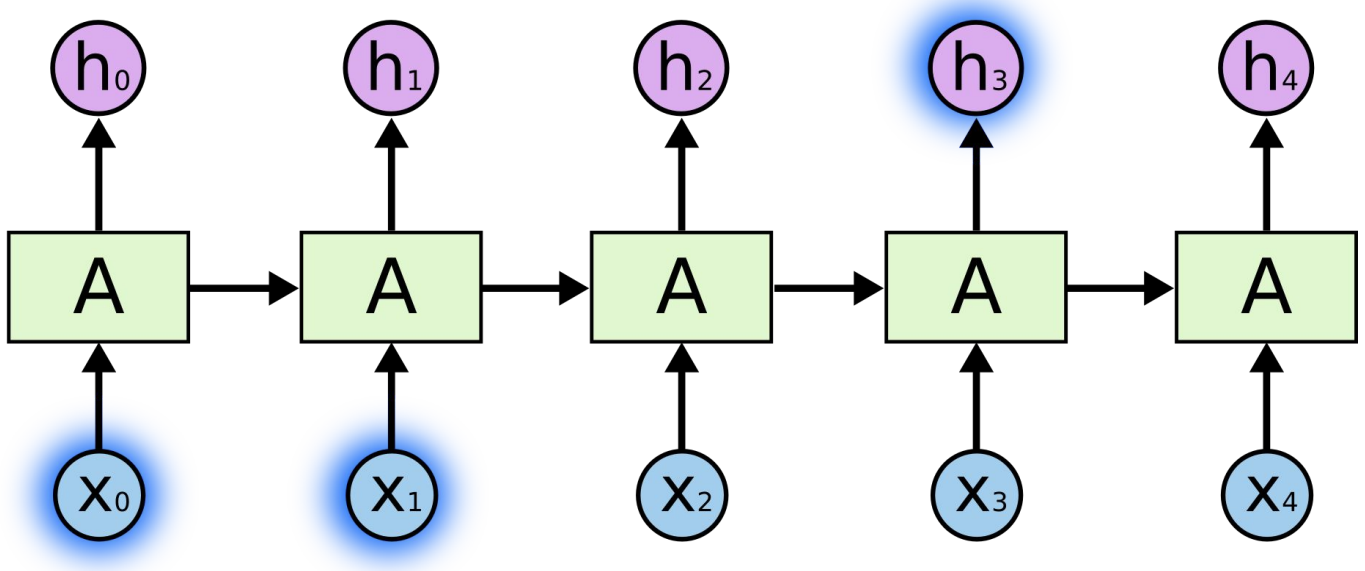


Redes Recorrentes (grafo de computação)...



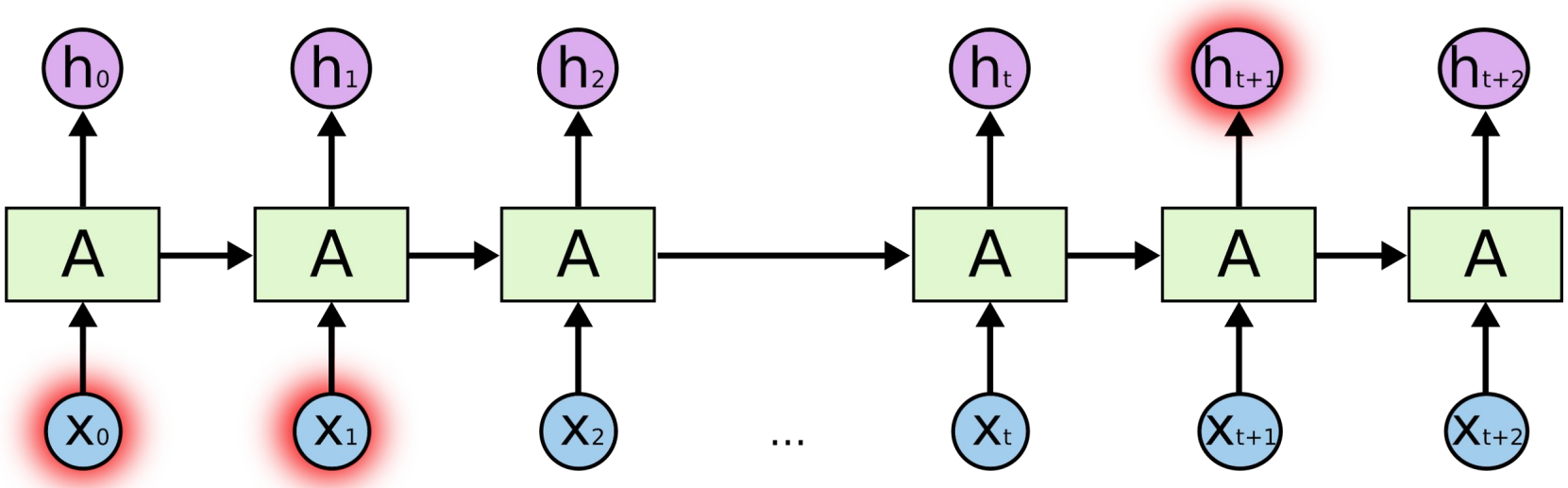
Redes Recorrentes… o problema!

- Redes recorrentes conseguem fazer isso (passar informação)?
  - Depende…
- Se precisamos só de informação recente, sim!
  - ex.: prever a última palavra de “as nuvens estão no *céu*”.
  - A distância entre informação relevante e o resto do contexto da frase é pequena.
  - RNN *pode* aprender como usar a informação prévia.



Redes Recorrentes… o problema!

- Redes recorrentes conseguem fazer isso (passar informação)?
  - Depende…
- Se precisamos de mais contexto… não!
  - ex.: prever a última palavra de “Eu nasci no Brasil... Sendo desse país todo esse tempo, falo fluentemente *português*”.
  - A distância entre informação relevante e o ponto onde ela é necessária pode ser grande.
  - Quando essa distância cresce, RNN pode não conseguir aprender a conectar essas informações.



Redes Recorrentes… o problema!

- Na teoria…
  - RNN consegue capturar essas relações distantes.
- Na prática…
  - Não consegue aprender, pois a RNN [1]:
    - tem que armazenar informação por uma duração arbitrária.
    - tem que ser resistente a ruído.
    - parâmetros da rede tem que ser treináveis (em tempo razoável)

[1] Y. Begio, Et al. *Learning long-term dependencies with gradient descent is difficult.* IEEE Trans. on Neural Networks. 1994

Redes Recorrentes… LSTM a solução!

- LSTM - *Long Short Term Memory*
  - Tipo especial de rede recorrente.
  - Projetada para aprender dependências de longo prazo [1].
  - Muito usada/adaptada [2]!
  - Inspiração para outras estratégias.
  - Inclui elementos adicionais para esquecer/lembrar de acordo com a relevância.

[1] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory.* Neural Computation 9(8). 1997
[2] J. Du. et al. *Novel Efficient RNN and LSTM-Like Architectures: Recurrent and Gated Broad Learning Systems and Their Applications for Text Classification.* IEEE Trans. on Cybernetics. 2020

Redes Recorrentes… LSTM a solução!

- LSTM - *Objetivo*
  - Lembrar/focar no que é mais importante
- Exemplo:

Cheguei em casa e troquei um disjuntor de energia que havia queimado por soltarem pipa no dia anterior e explodirem o transformador. Agora consigo ligar o computador.

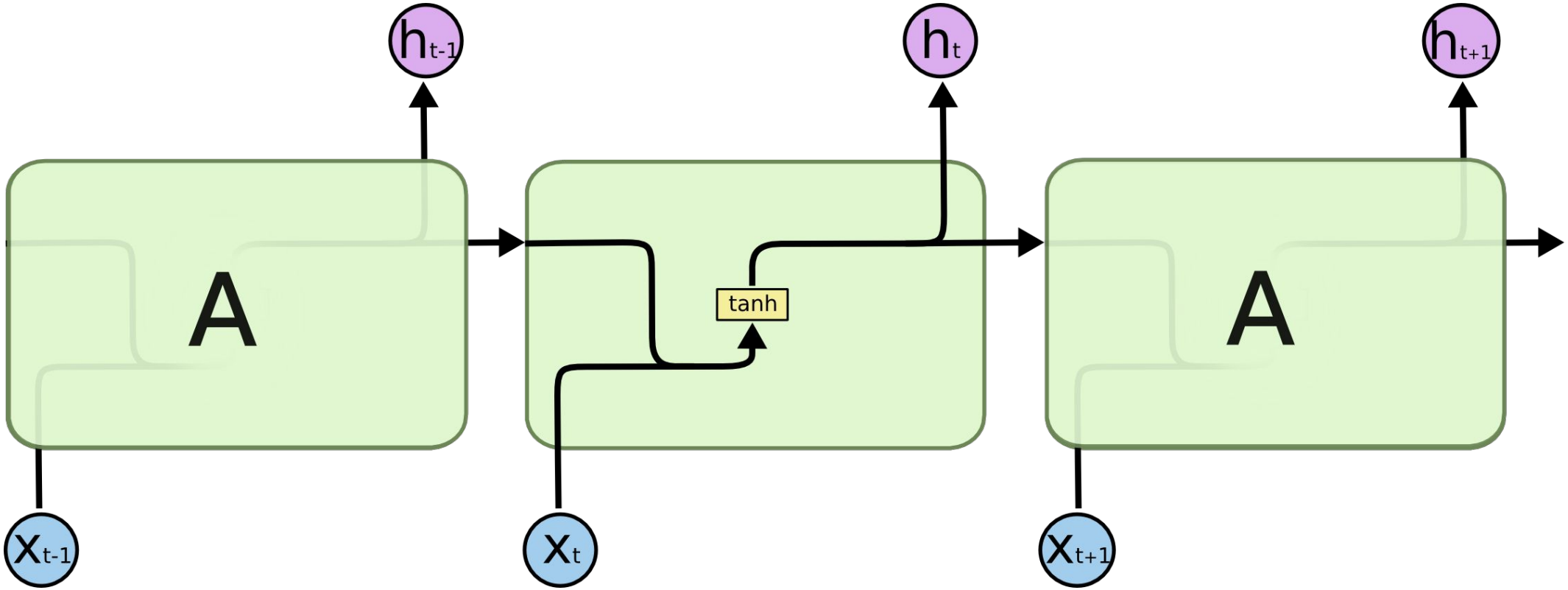
Redes Recorrentes… LSTM a solução!

- LSTM - *Objetivo*
  - Lembrar/focar no que é mais importante
- Exemplo:

Cheguei em casa e troquei um disjuntor de energia que havia queimado por soltarem pipa no dia anterior e explodirem o transformador. Agora consigo ligar o computador.

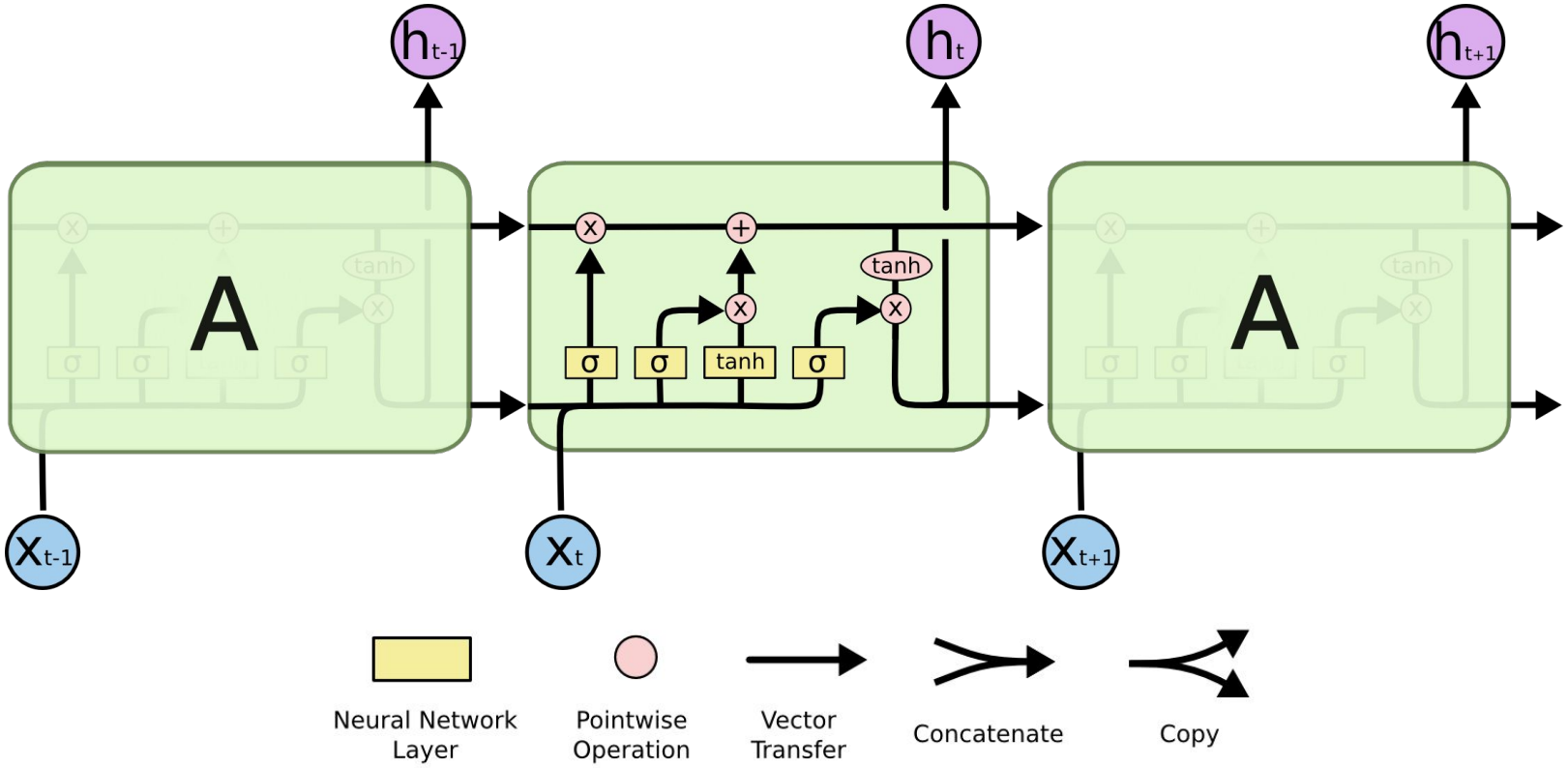
RNN vs LSTM

- Internamente, as Redes Neurais Recorrentes têm uma estrutura simples com uma cadeia de repetição sobre a camada (função tangente hiperbólica):



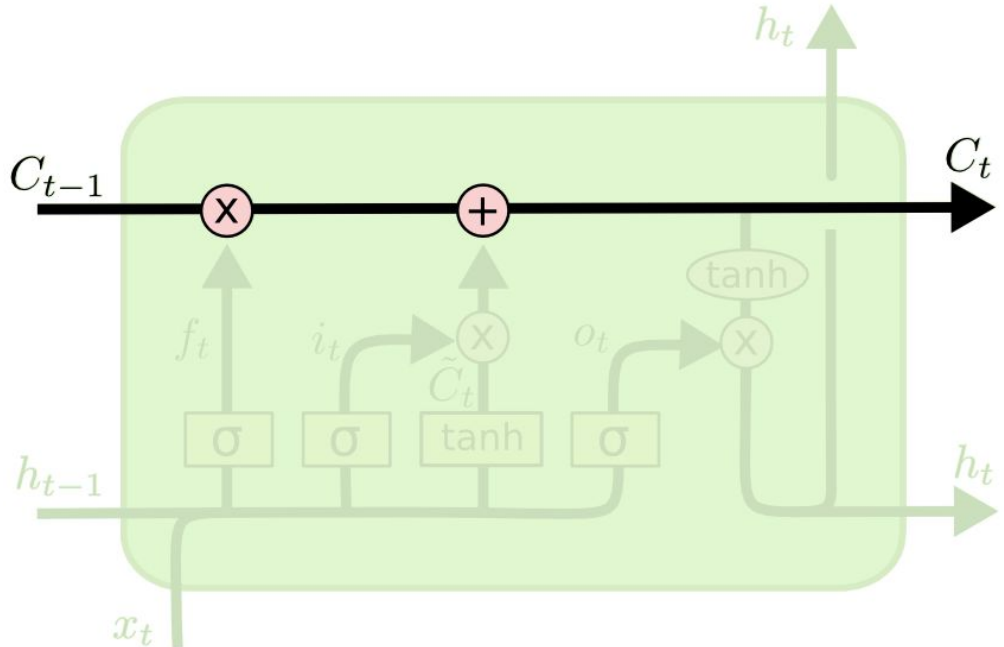
RNN vs LSTM

- LSTM também têm essa cadeia de repetição, com elementos adicionais na estrutura.
    - GATES!
    - Ao invés de uma única camada de rede neural, temos quatro:



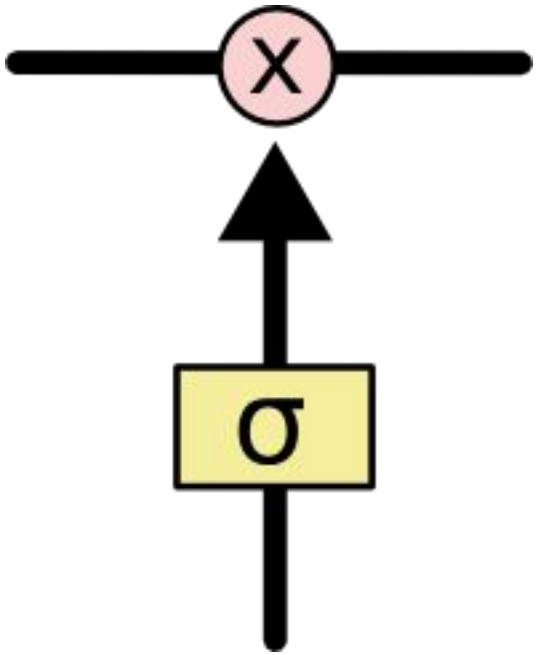
LSTM - Idéia Central

- A idéia central do LSTM é a manipulação estado da célula (topo do diagrama)
  - Como uma “esteira” que passa por toda estrutura interna.
    - Fácil a informação passar por ela com poucas modificações (poucas interações).
    - LSTM pode adicionar/remover informações a partir de interações com suas gates.
  - Visa tratar o problema da explosão/sumiço dos gradientes:
    - Conexão direta sem função de ativação -> gradiente flui sem derivações adicionais.



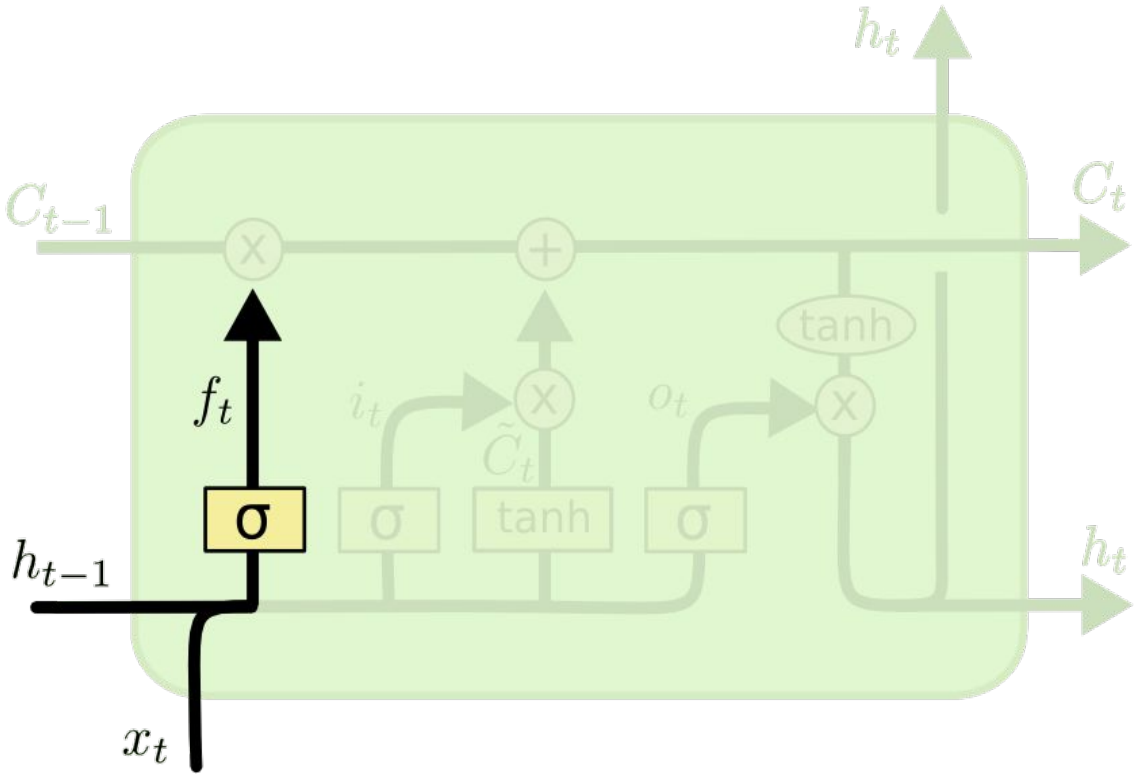
LSTM - Idéia Central

- As gates são uma maneira de deixar a informação passar.
  - Compostas de uma camada com função de ativação sigmoid e uma operação de multiplicação.
  - Sigmoid retorna um número entre 0 e 1 (quanto da informação vai passar).
  - LSTM tem quatro dessas gates.



LSTM - Passo-a-passo - esquecer (forget gate)

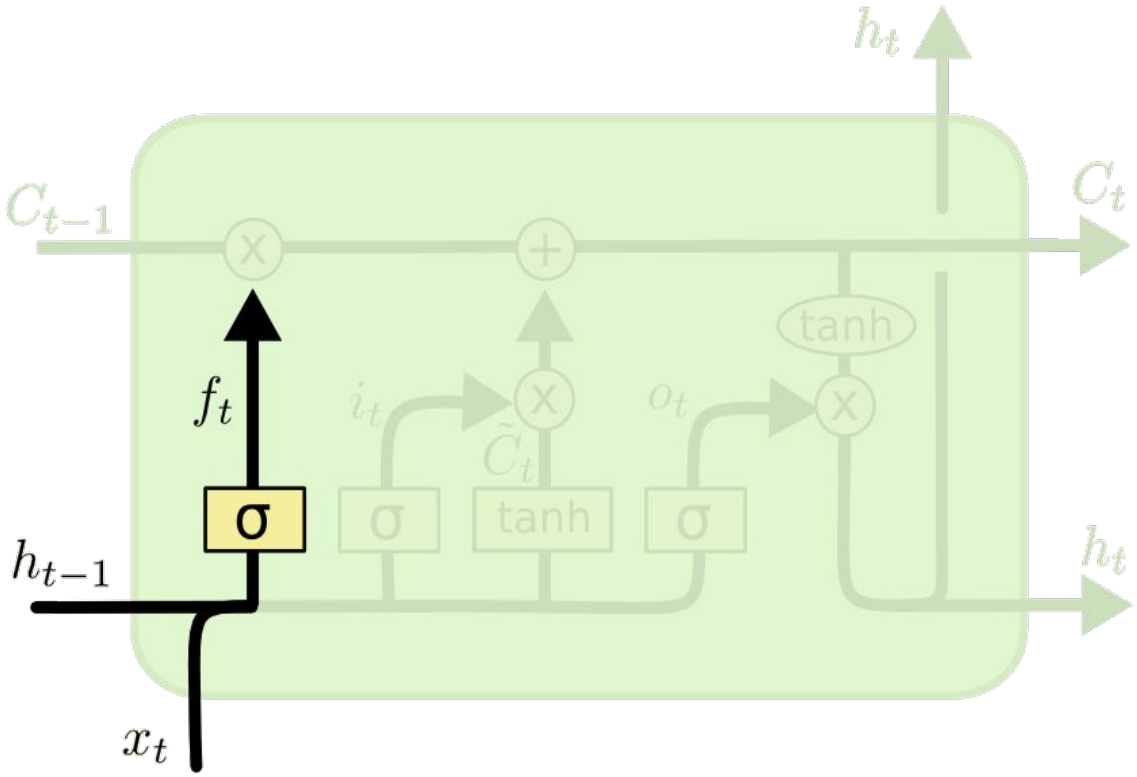
- Primeiro passo é decidir qual informação de estado jogar fora (forget gate).
  - Essa decisão é tomada na porta que decide o que esquecer (abaixo).
  - olha para h_{t-1} e x_t e a saída é um número entre 0 e 1
    - 0 - esquece, 1 - lembra tudo



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

LSTM - Passo-a-passo - esquecer

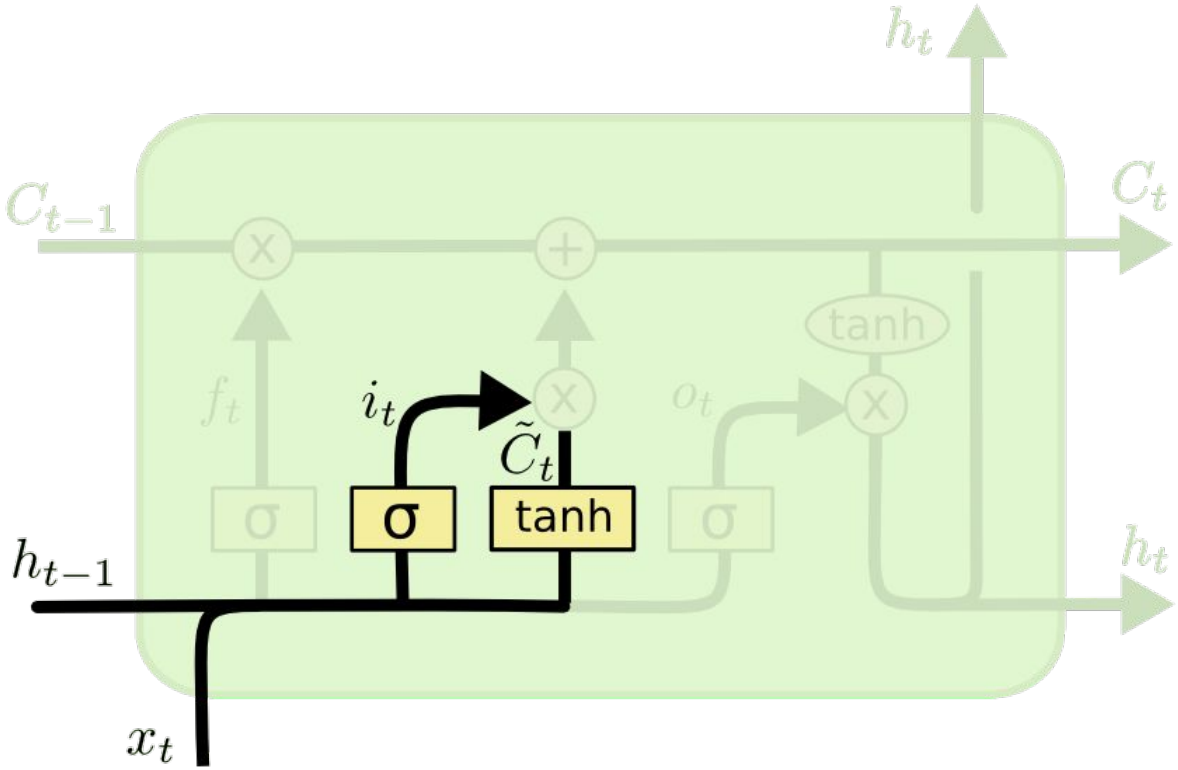
- Primeiro passo é decidir qual informação de estado jogar fora.
  - Ex.: quando passando pela frase, eu quero lembrar a palavra “não” que nega a próxima. Depois, para processar as outras, quero esquecer a negação.
    - “**Não** gosto de uva mas eu gosto de maçã”



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

LSTM - Passo-a-passo - adicionar

- Depois quero decidir qual informação da entrada vou armazenar.
  - Duas gates:
    - tanh (cadidate gate layer) - candidatos a serem somados/adicionados ao estado atual.
    - sigmoid (input gate layer) - “peso” das informações que quero adicionar.

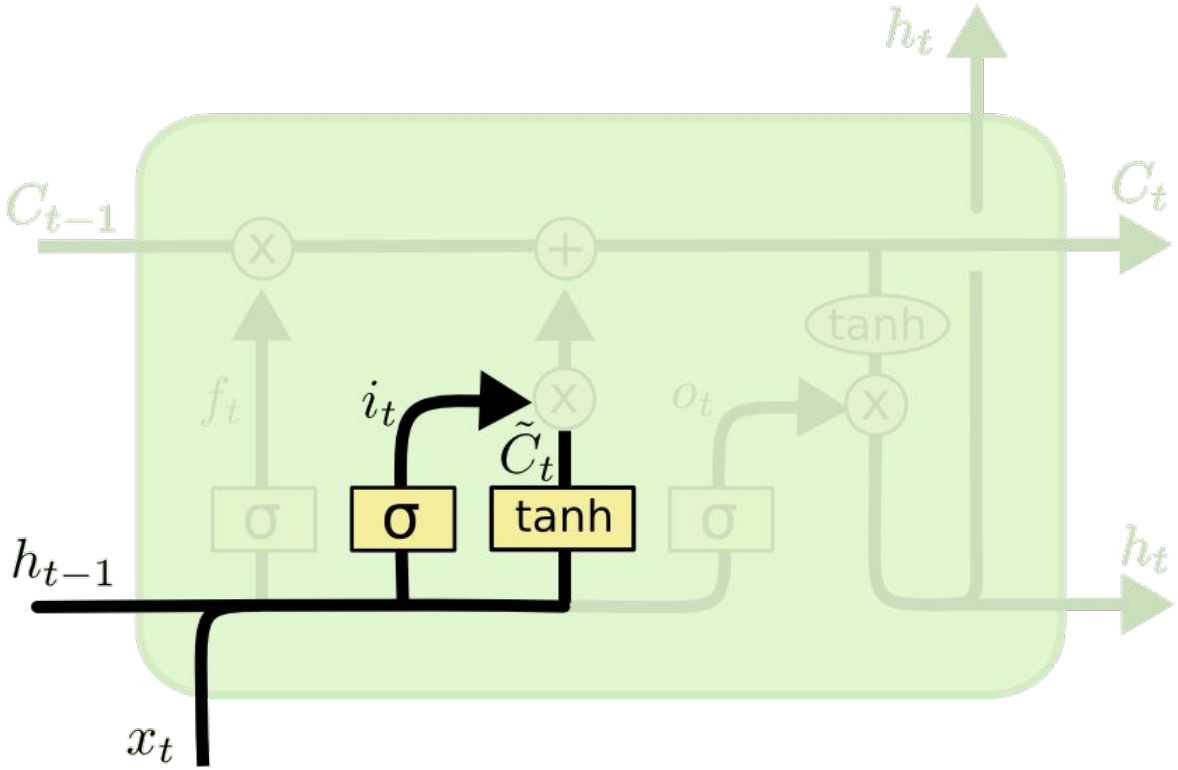


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

LSTM - Passo-a-passo - adicionar

- Depois quero decidir qual informação vou armazenar (atualizar estado C_{t-1}).
  - Ex.: estou processando a palavra “ela” na sequência de: “(...) **ele** é o **marido**, **ela** é a **esposa.**”
  - Antes de adicionar o “ela” em Ct, pondero a nova informação de estado (“ela”) com i_t considerando informações do velho estado (ex. "ele"), para **adicionar** a informação exclusiva do “ela” com $i_t * C_t$.

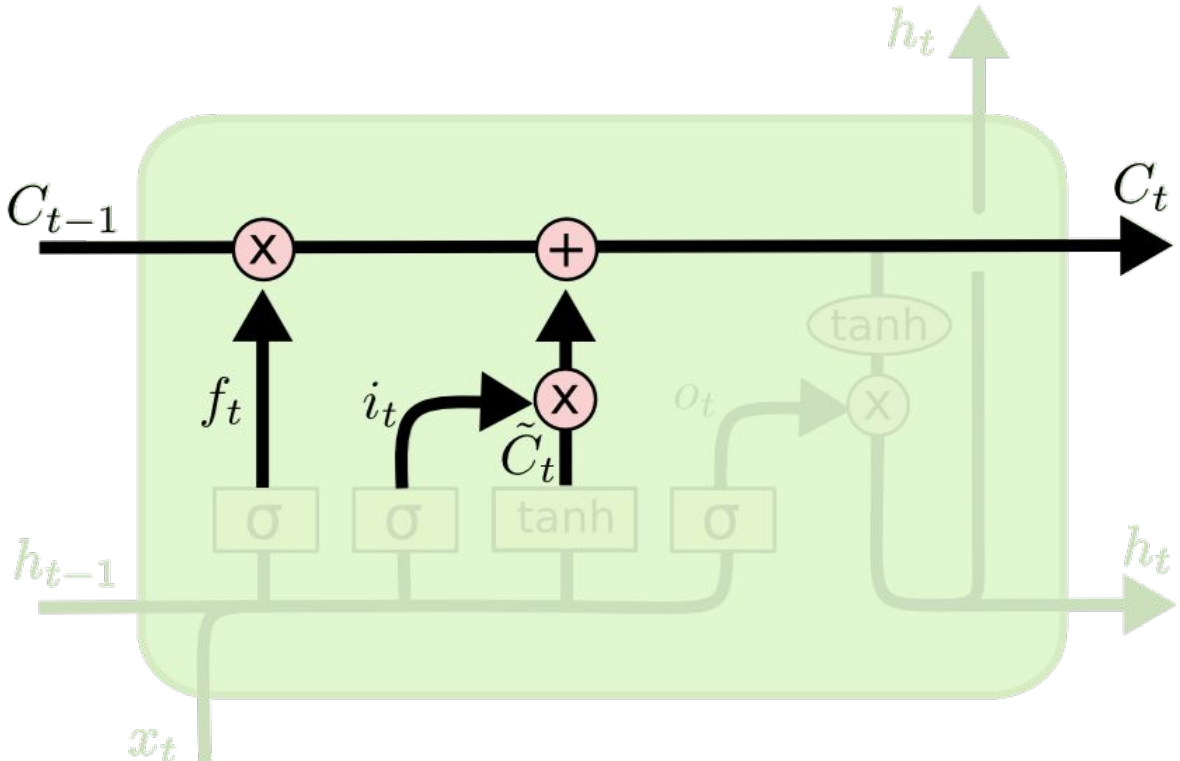


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

LSTM - Passo-a-passo - adicionar

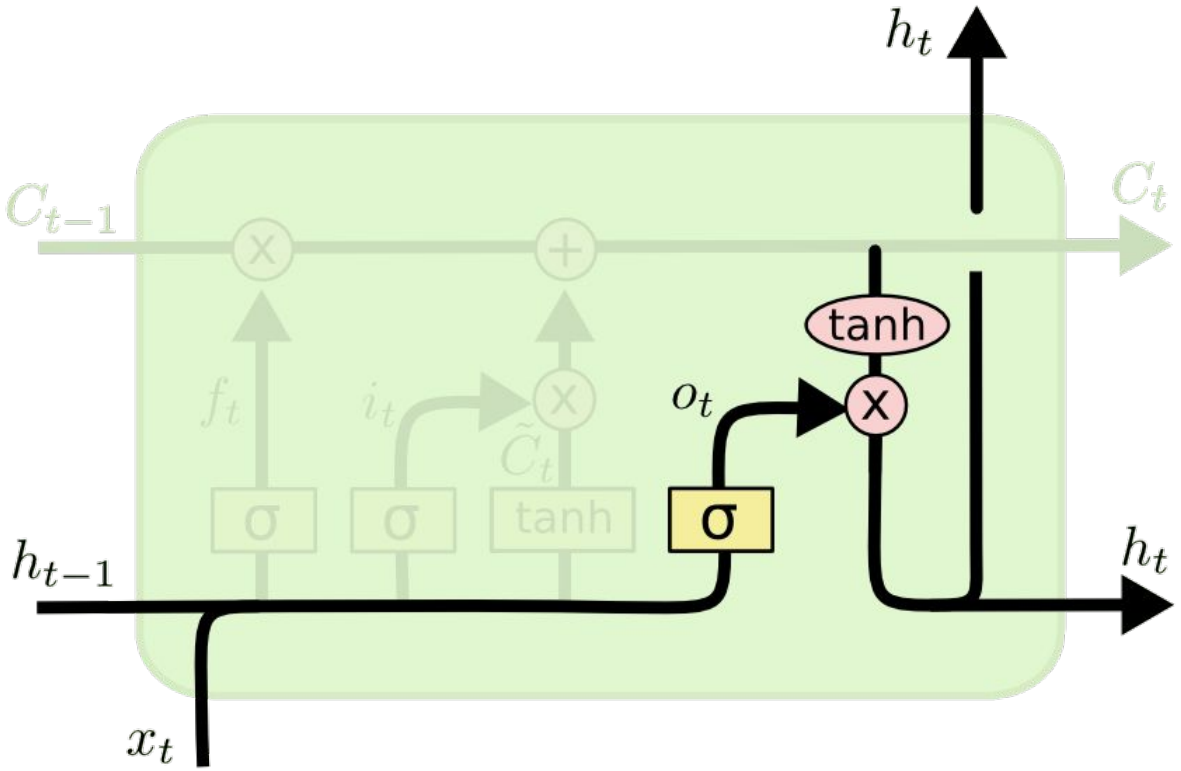
- Atualizar o estado anterior C_{t-1} para o novo estado C_t
  - Passos anteriores já decidiram o que fazer!
    - Multiplica o estado anterior pelo que devemos esquecer do estado anterior $f_t * C_{t-1}$
    - Adiciona pelo que queremos armazenar do novo estado $i_t * C_t$.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM - Passo-a-passo - prever (output gate)

- Finalmente, decidir qual será a saída:
  - Predição final:
    - Primeiro, a sigmoid o quanto da memória C_t será exposta no estado oculto h_t
    - depois, utiliza o tanh para gerar as saídas da predição, que serão multiplicadas pelo o_t

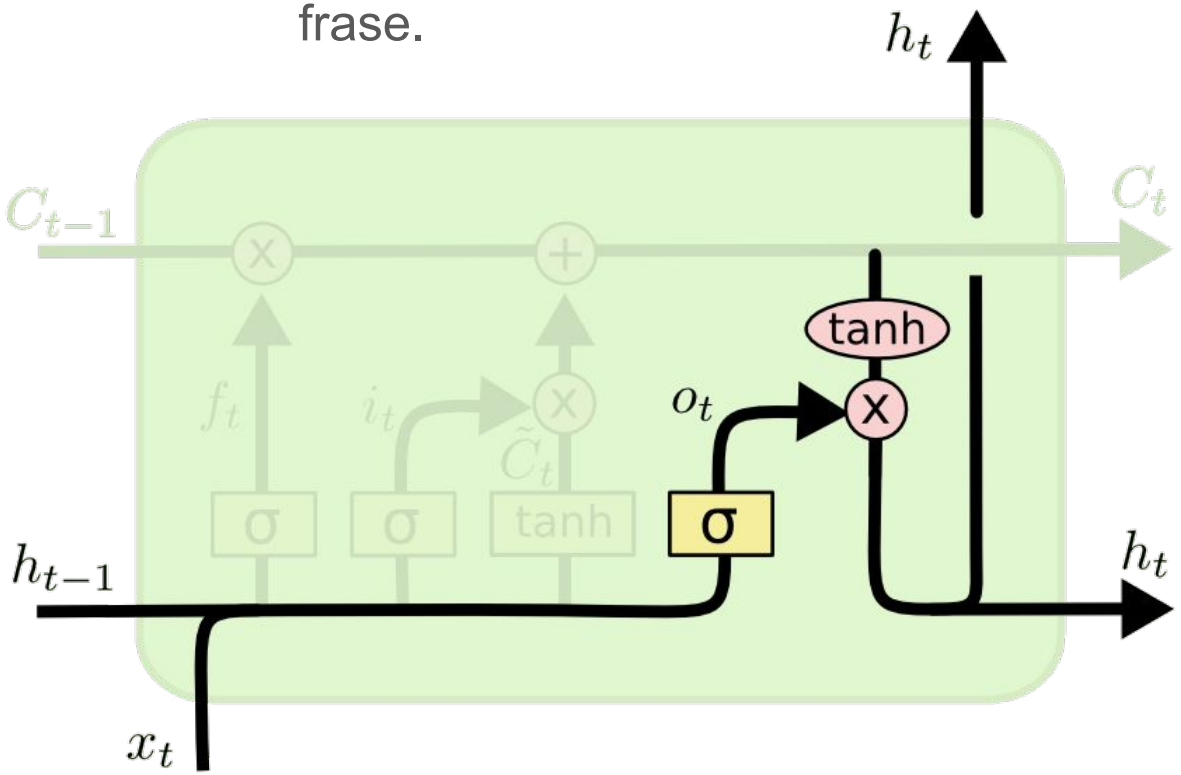


$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

LSTM - Passo-a-passo - prever

- Finalmente, decidir qual será a saída:
  - Por exemplo, seja a tarefa de prever a classe gramatical das palavras.
  - Dado um texto, podermos ter passado pelo sujeito. Logo, queremos prever que a próxima palavra deve ser um verbo. Para isso, usamos ht como features para um classificador.
  - A saída (verbo) deve representar um verbo conjugado no singular ou plural com base no sujeito da frase.



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

LSTM

- Vantagens:
  - Trata vanishing/exploding.
    - Cell State força preservar gradientes em diferentes pontos do tempo.
  - Trata dependências de longo prazo
    - Forget gate pode ignorar informações intermediárias
  - Trata ruído.
    - Camadas sigmoid podem remover informações irrelevantes
- Desvantagens:
  - 4 gates diferentes (4x mais parâmetros que uma RNN tradicional)
    - mais lento
    - mais difícil “sincronizar” o funcionamento de todos os gates via backpropagation.
      - Não tem a noção do “todo” como nas CNNs ou MLP.
  - Treino sequencial (como nas RNNs)

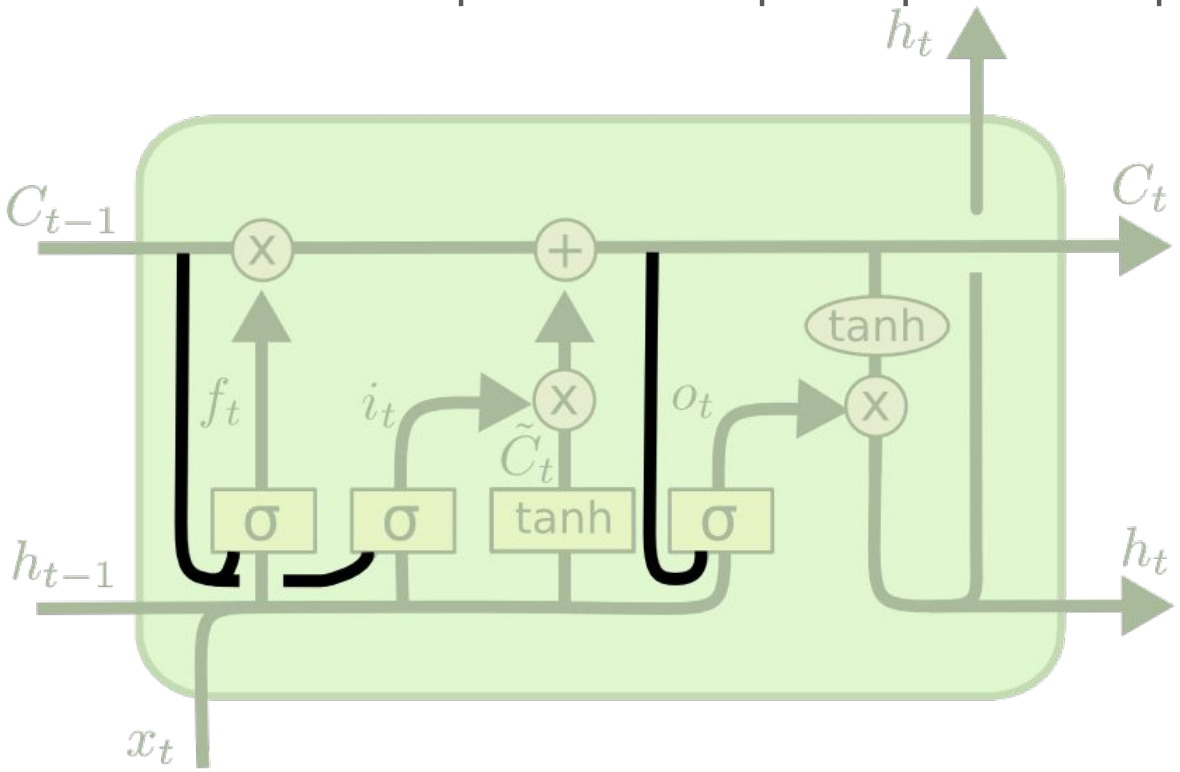
LSTM - Variações

- Apesar do LSTM básico descrito anteriormente, há múltiplas variações do LSTM literatura baseadas em:
  - Adicionar novas conexões internas [1,3].
  - GRU (Gated Recurrent Unit) [2,3,4,5]
  - Attention Networks [3]

[1] F. Gers, J. Schimibhuber. Recurrent Nets that Time and Count. IEEE-INNS. 2000
[2] K. Cho. Et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. EMNLP. 2014
[3] Yu, Shujuan et al. 'Attention-based LSTM, GRU and CNN for Short Text Classification'. 1 Jan. 2020 : 333 – 340.
[4] M. Zulqarnain. Efficient processing of GRU based on word embedding for text classification. JOIV. 2019
[5] N., Vahid, et al. A bi-directional cross-channel RNN model for time-series forecasting of dairy production. Scientific Reports. 2025.

LSTM - Adicionar Novas Conexões

- Permite que as camadas internas vejam o estado da célula:
  - Aprende distinção entre sequências entre múltiplos tics de tempo.
  - Útil para tarefas que requerem tempo (e sua contagem).



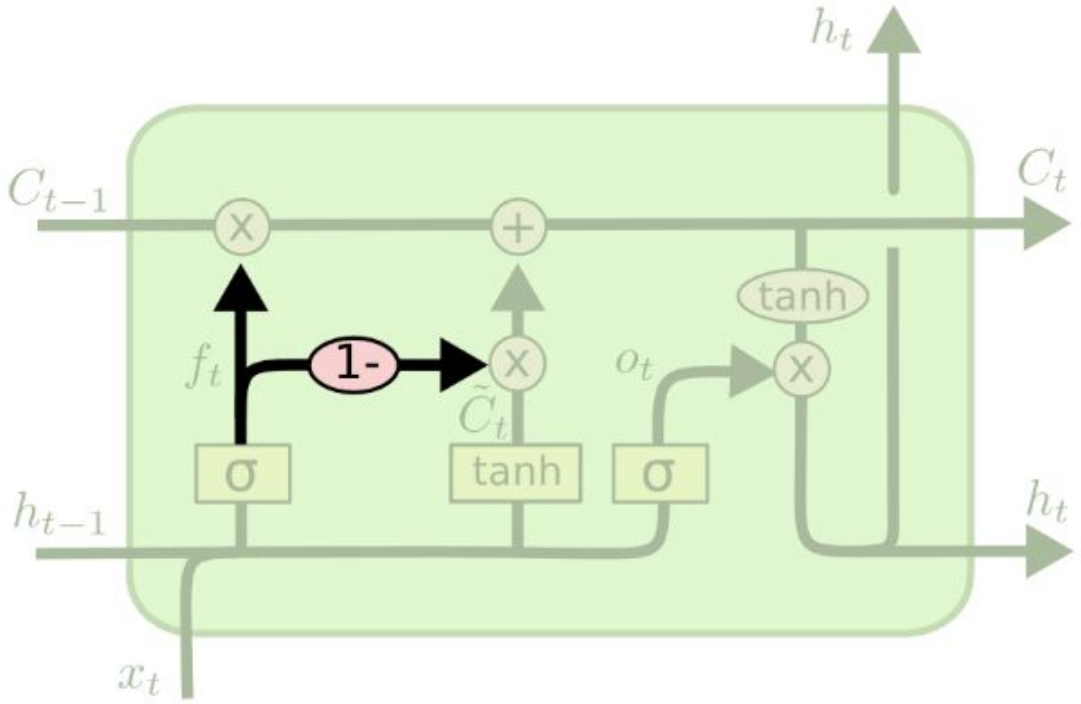
$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$

$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$

$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

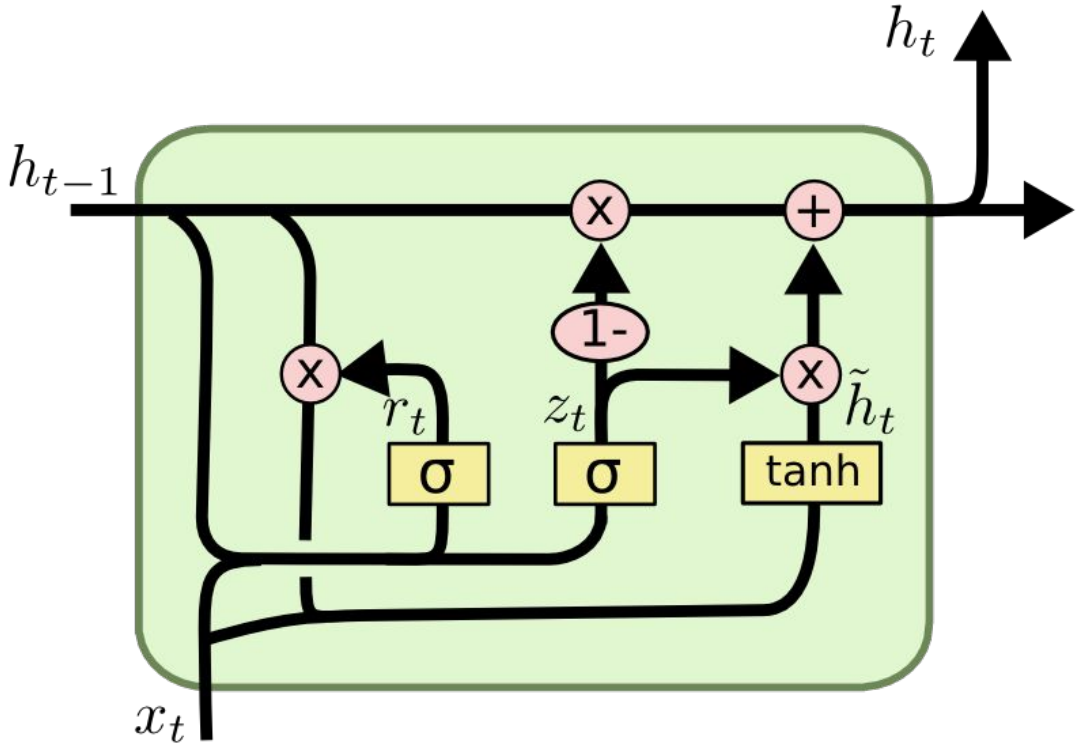
LSTM - Adicionar Novas Conexões

- Só esquece quando vai colocar algo no lugar:



GRU - Gated Recurrent Unit

- Elimina o estado da célula (C_t) e usa somente o estado oculto (h_t).
  - Similar à LSTM - idéia central de esquecer/manter informação.
  - Gates de atualização e esquecimento.
  - Usa somente o estado oculto para repassar informação.
  - Menos operações = Treino mais rápido que LSTM.



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

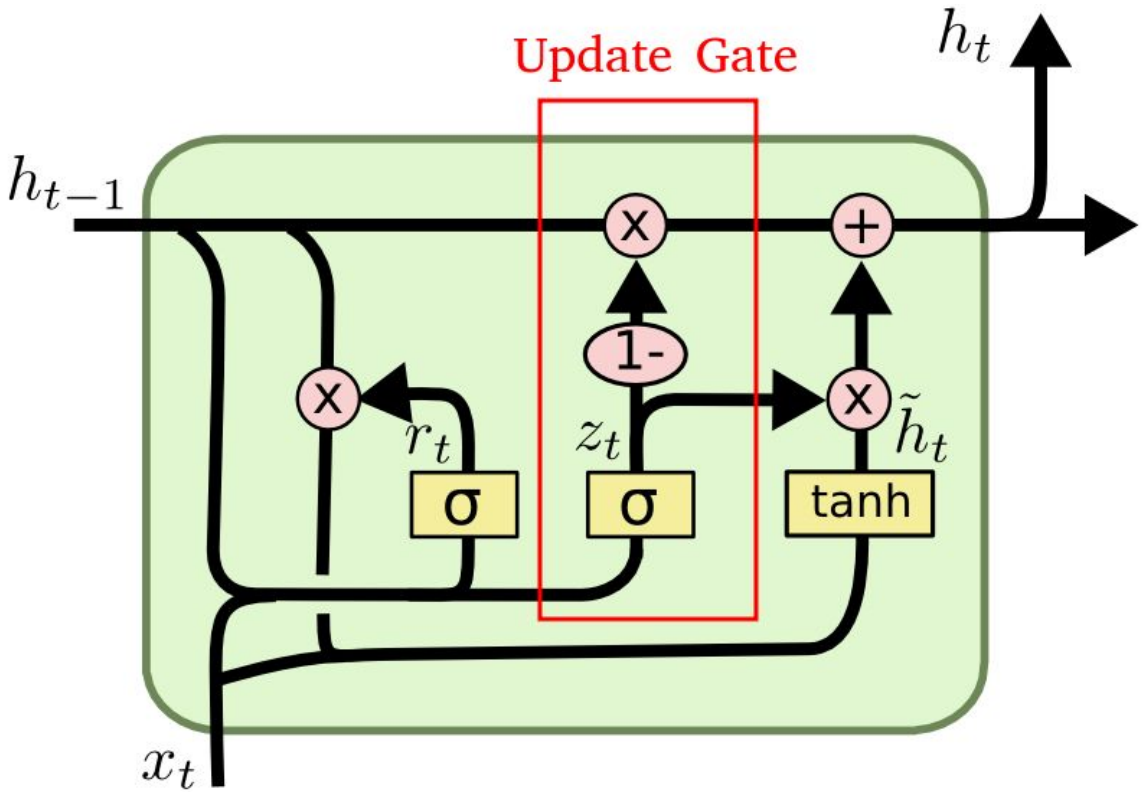
$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

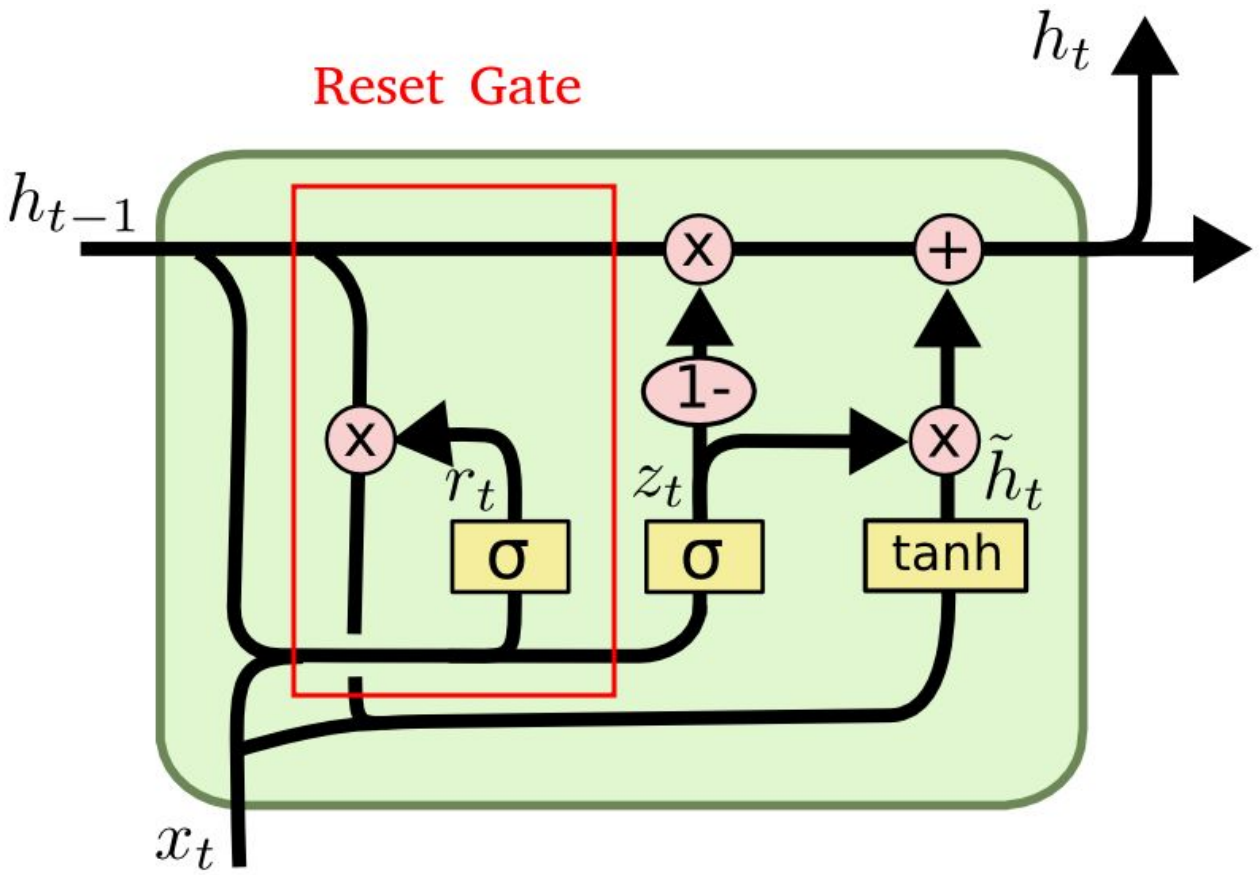
GRU- Gated Recurrent Unit

- Update Gate
  - Qual informação atual jogar fora e qual informação adicionar.
  - Combina input e forget gate do LSTM
    - se eu não esqueço, não entra informação



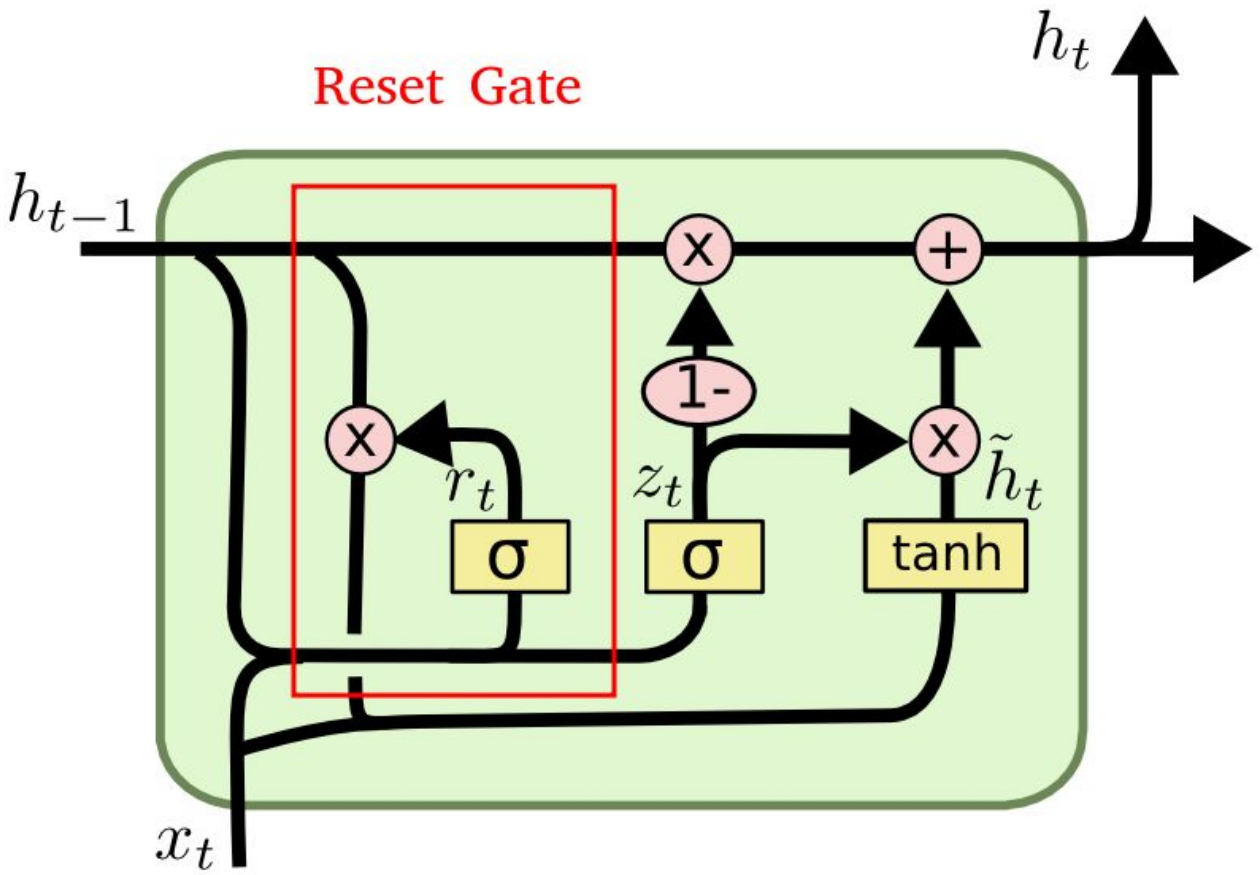
GRU- Gated Recurrent Unit

- Reset Gate
  - Quanto da informação anterior devo esquecer.



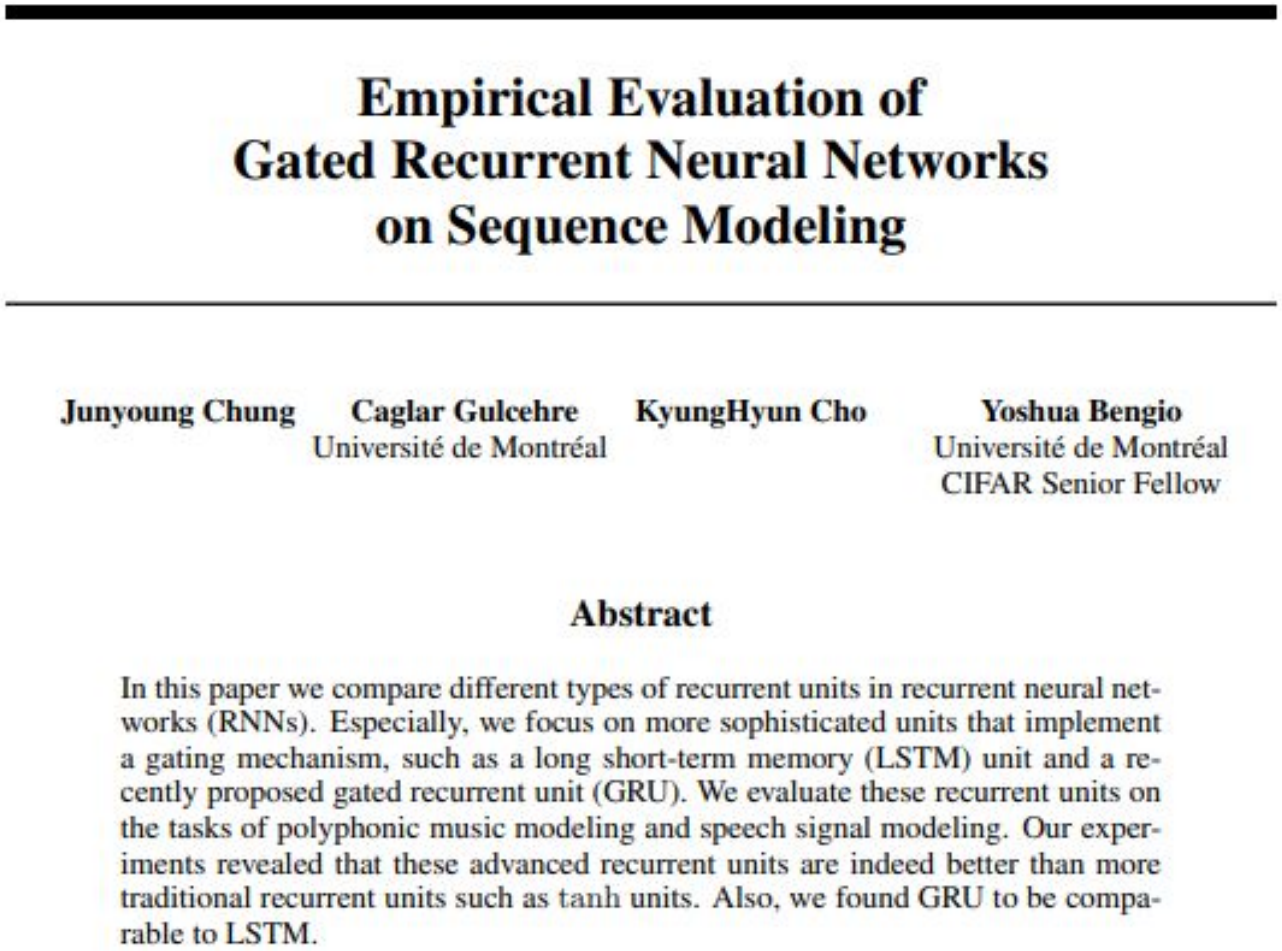
GRU- Gated Recurrent Unit

- Reset Gate
  - Quanto da informação anterior devo esquecer.



Qual arquitetura devo usar?

- Eficiência:
  - RNN: sequência muito pequena, pouco ruído, pouco poder computacional
  - GRU: Mais eficiente que LSTM, resultados parecidos
- LSTM vs GRU
  - Boa pergunta!



Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling

Junyoung Chung **Caglar Gulcehre** **KyungHyun Cho** **Yoshua Bengio**
Université de Montréal
Université de Montréal
CIFAR Senior Fellow

Abstract

In this paper we compare different types of recurrent units in recurrent neural networks (RNNs). Especially, we focus on more sophisticated units that implement a gating mechanism, such as a long short-term memory (LSTM) unit and a recently proposed gated recurrent unit (GRU). We evaluate these recurrent units on the tasks of polyphonic music modeling and speech signal modeling. Our experiments revealed that these advanced recurrent units are indeed better than more traditional recurrent units such as tanh units. Also, we found GRU to be comparable to LSTM.

LSTM - GRU: Implementação

```
tf.keras.layers.LSTM(
    units,
    activation="tanh",
    recurrent_activation="sigmoid",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    unit_forget_bias=True,
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    time_major=False,
    unroll=False,
    **kwargs
)
```

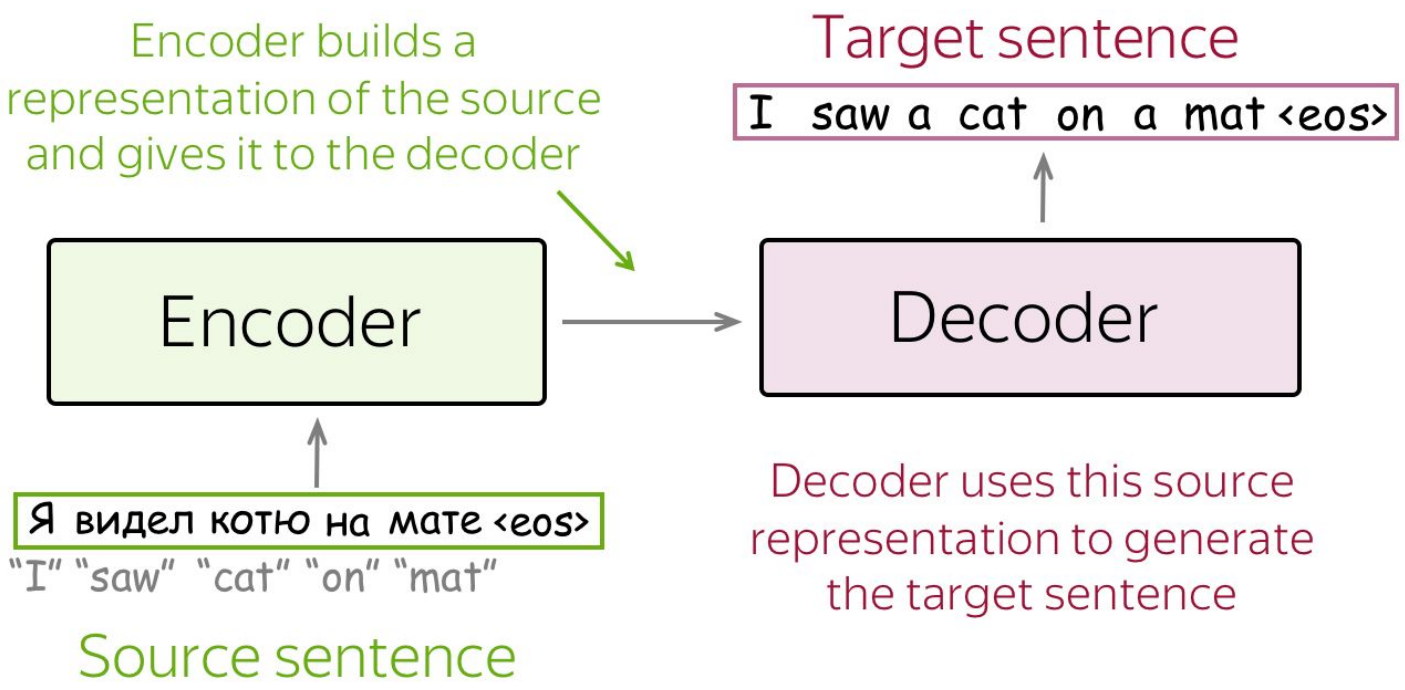
LSTM - GRU: Implementação

```
embedding_vecor_length = 32
model = Sequential()
model.add(Embedding(top_words, embedding_vecor_length, input_length=max_review_length))
model.add(LSTM(100))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
print(model.summary())
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=3, batch_size=64)
```

fonte:
https://machinelearningmastery.com/sequence-classification-lstm-recurrent-neural-networks-python-keras/

Seq2seq com redes recorrentes

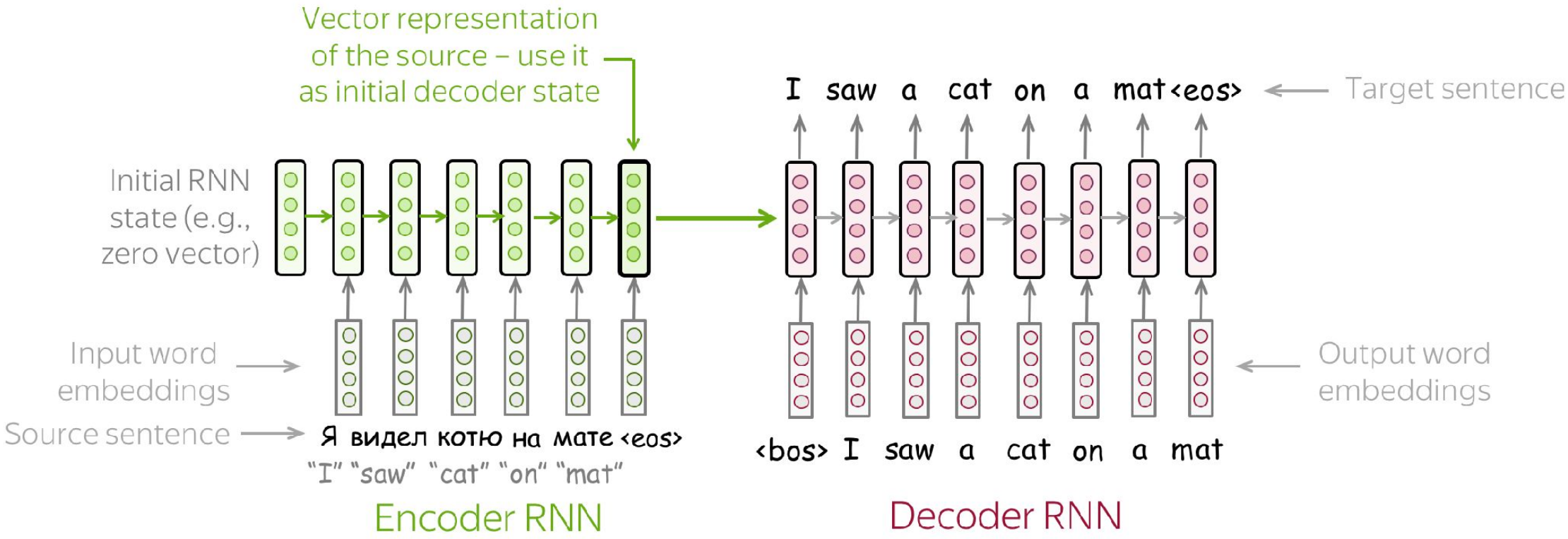
Entrada: $x_1, \ldots, x_T$. Saída: $y_1, \ldots, y_{T'}$. Os comprimentos podem diferir.



Encoder–decoder com RNN

Arquitetura clássica: Sutskever, Vinyals e Le (2014). https://arxiv.org/abs/1409.3215

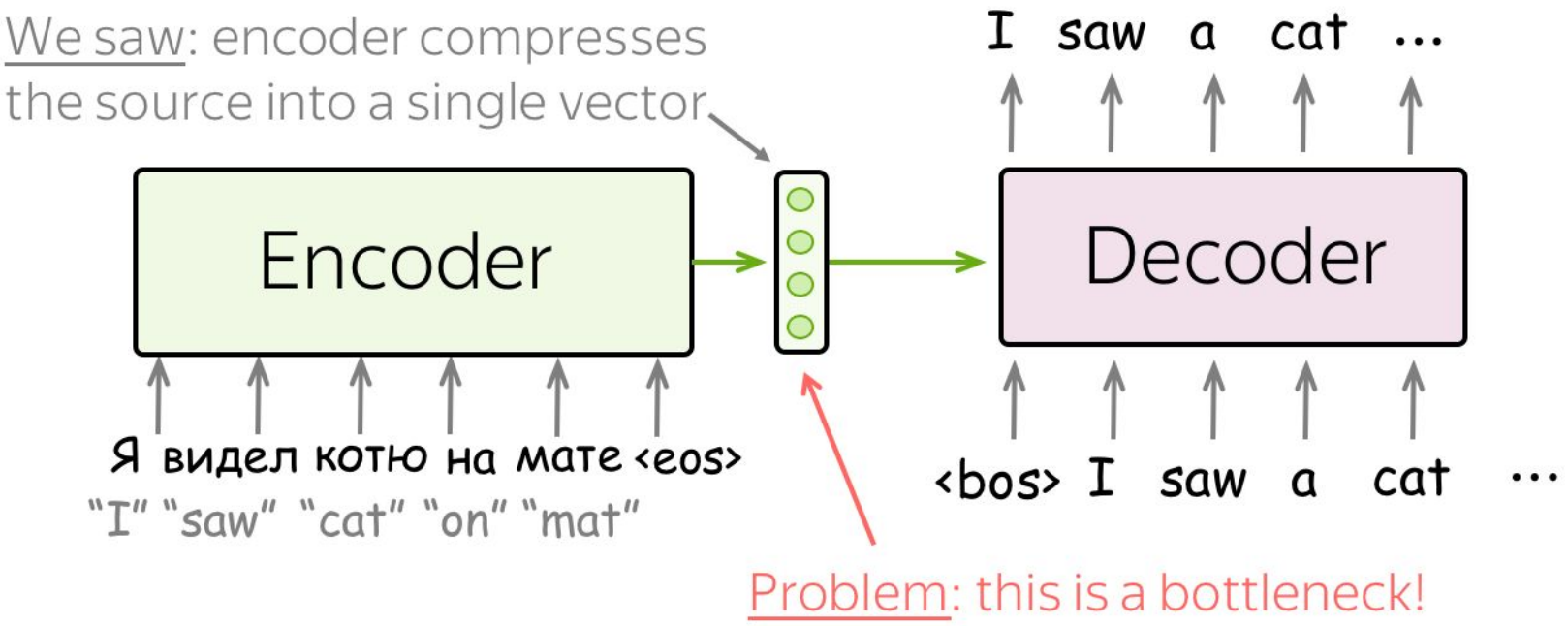
- O encoder lê a entrada e atualiza sua memória.
- O decoder inicia com o estado final e gera a saída.



Encoder e decoder aprendem conjuntos próprios de pesos.

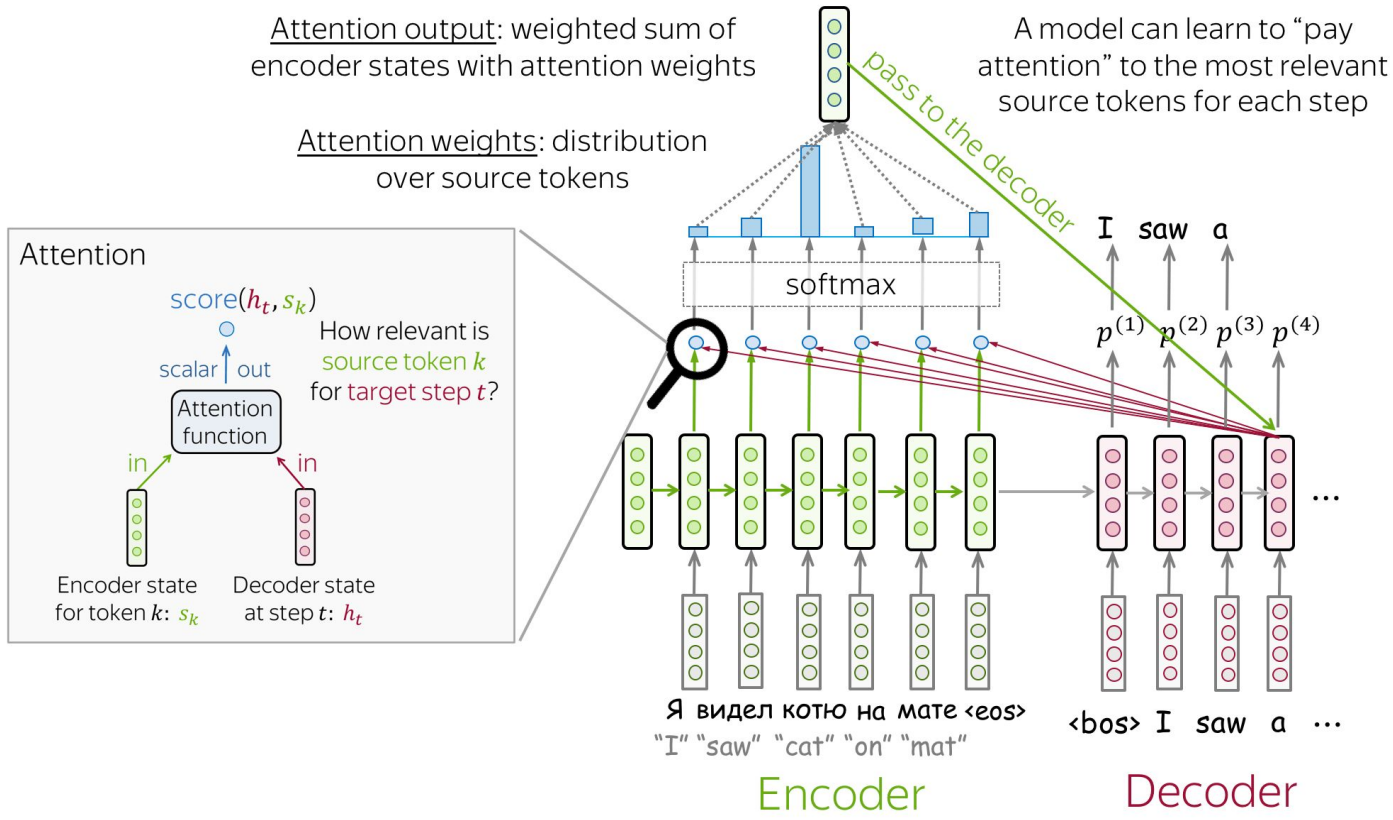
Gargalo da representação fixa

- Toda a entrada precisa caber em um único vetor.
- Sequências longas dificultam a retenção dos detalhes.
- Cada palavra gerada pode precisar de uma parte diferente da entrada.



Seq2seq com atenção e RNN

- O decoder consulta todos os estados do encoder.
- A atenção produz um contexto específico para cada passo.



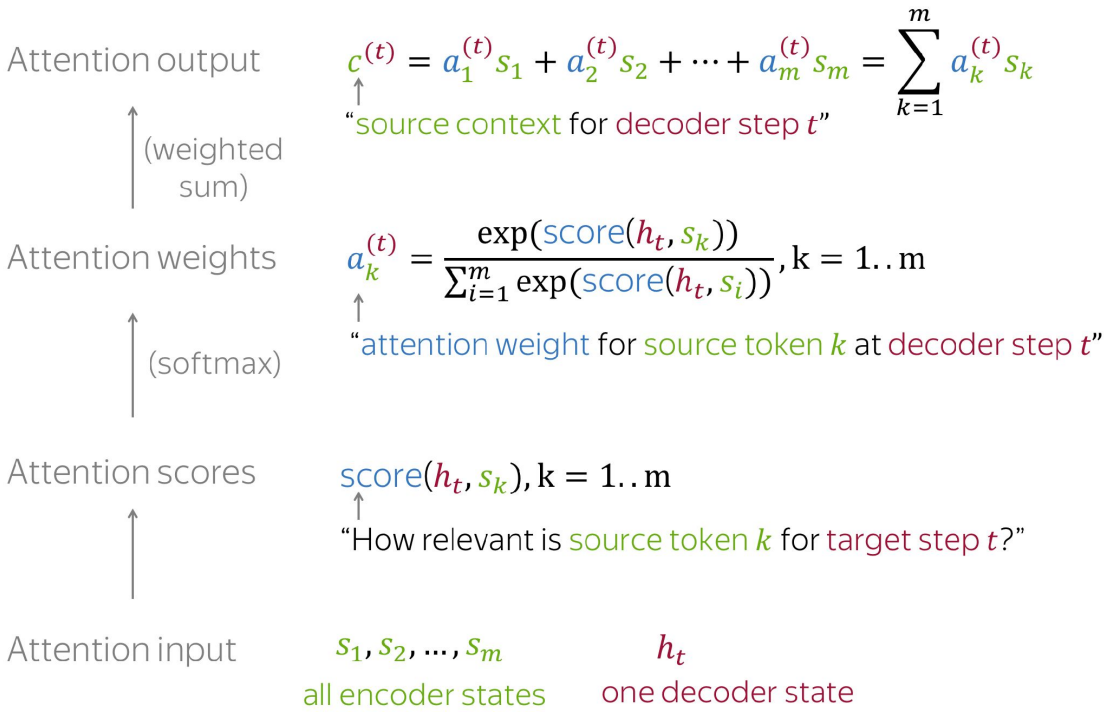
Bahdanau, Cho e Bengio, Neural Machine Translation by Jointly Learning to Align and Translate (ICLR 2015) https://arxiv.org/abs/1409.0473

Cálculo da atenção

Score: $e_{□i} = score(h_{□-1}, s_i)$

Pesos: $\alpha_{□i} = \exp(e_{□i}) / \Sigma_{□} \exp(e_{□□})$

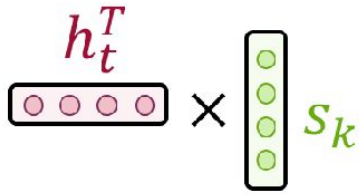
Contexto: $a_{□} = \Sigma_i \alpha_{□i} s_i$



Funções de score

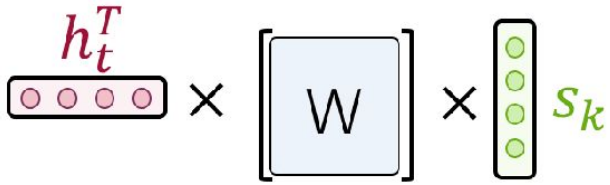
Produto escalar: mede “similaridade" entre representações.

Dot-product



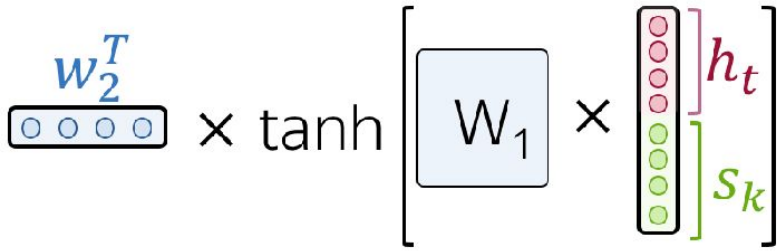
$\text{score}(h_t, s_k) = h_t^T \; s_k$

Bilinear



$\text{score}(h_t, s_k) = h_t^T \; W s_k$

Multi-Layer Perceptron



$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1[h_t, s_k])$

Referências

- François Chollet. Deep Learning with Python. Manning publications. 2019 (cap 6)
- Aurélien Géron. Neural networks and deep learning https://www.oreilly.com/library/view/neural-networks-and/9781492037354/ch01.html - Excelente material teórico!

Recursos Interessantes:

- S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory.* Neural Computation 9(8). 1997
- K. Cho. Et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. EMNLP. 2014
- http://colah.github.io/posts/2015-08-Understanding-LSTMs/